



UNIVERSIDAD DEL BÍO-BÍO
FACULTAD DE CIENCIAS EMPRESARIALES
ESCUELA INGENIERÍA CIVIL INFORMÁTICA

PROYECTO DE TÍTULO

“Análisis comparativo de modelos de Deep Learning basados en YOLO, para la tarea de detección de lugares disponibles, en estacionamientos del campus Fernando May de la Universidad del Bío-Bío”

PARA OBTENER EL GRADO ACADÉMICO DE
INGENIERO CIVIL EN INFORMÁTICA

AUTORES

Rodrigo Ignacio Carrasco Cofré
José Patricio Rivera Medina

PROFESORA GUÍA

María Antonieta Soto Chico

PROFESORA CO-GUÍA

Carola Andrea Figueroa Flores

CHILLÁN, 15 DE ENERO DE 2023

“Nunca debes rendirte a la desesperación. Si te permites ir por ese camino, te rendirás a tus instintos más bajos. En tiempos oscuros, la esperanza es algo que te das a ti mismo. Ese es el significado de la verdadera fuerza interior”

Resumen

El proyecto titulado “Análisis comparativo de modelos de Deep Learning basados en YOLO, para la tarea de detección de lugares disponibles, en estacionamientos del campus Fernando May de la Universidad del Bío-Bío” se entrega para cumplir con los requerimientos demandados por la Universidad del Bío-Bío para la obtención del título universitario de la carrera Ingeniería Civil en Informática.

La gestión de estacionamientos se ha vuelto un problema creciente debido a que las áreas urbanas se encuentran en constante crecimiento, cada vez hay más vehículos circulando, por lo tanto, la cantidad de estacionamientos disponibles no satisface la demanda. En consecuencia, los conductores pasan una gran cantidad de tiempo buscando algún lugar disponible. Esto genera una serie de problemas dentro de los que destacan el estrés, la contaminación, la pérdida de tiempo y la pérdida de combustible.

La presente investigación, tiene como propósito evaluar y contrastar distintos tipos de modelos pertenecientes a la arquitectura de red neuronal convolucional YOLO. El objetivo principal es identificar mejoras potenciales en la gestión actual de los estacionamientos de la Universidad del Bío-Bío por medio del uso de inteligencia artificial. Con este fin, se llevó a cabo una comparación entre diversas variantes de los modelos YOLOv5 y YOLOv8. Donde cada uno de estos modelos, fue sometido a entrenamiento utilizando dos conjuntos de datos diferenciados. Donde, el primer conjunto denominado "VehículosUBB", se centra específicamente en la detección de vehículos, mientras que el segundo, denominado "EspaciosUBB", se orienta hacia la identificación del estado de los espacios de estacionamiento, ya sea disponibles u ocupados.

Los resultados obtenidos revelan una disparidad significativa entre los modelos entrenados con el conjunto de datos "EspaciosUBB" en comparación con aquellos entrenados con el conjunto "VehículosUBB". Las métricas utilizadas reflejan un rendimiento sustancialmente superior en el caso de la detección de espacios de estacionamiento en contraste con la detección de vehículos. Este hallazgo sugiere una mayor eficacia y precisión en la capacidad predictiva de los modelos cuando se centran en la identificación de espacios disponibles u ocupados.

Adicionalmente, se presenta en este estudio la creación de un software destinado a destacar la aplicación práctica de las predicciones generadas por el modelo. Este software, ha sido concebido con una estructura sólida que otorga a los usuarios la capacidad de interactuar de manera efectiva con los datos predichos. Este enfoque práctico no solo resalta la utilidad de los modelos en el contexto específico de la Universidad del Bío-Bío, sino que también establece un precedente valioso para la implementación de soluciones similares en otros entornos.

La relevancia de estos resultados trasciende la mera eficiencia técnica, ya que tienen el potencial de impactar positivamente en la gestión y la calidad de vida dentro del campus universitario. La capacidad de optimizar la utilización de los espacios de estacionamiento puede tener beneficios tangibles en términos de flujo de tráfico, eficiencia logística y comodidad para la comunidad universitaria, contribuyendo así a una mejora sustancial en la experiencia cotidiana en el campus.

Abstract

This project titled "Comparative Analysis of Deep Learning Models Based on YOLO for the Task of Detecting Available Spaces in Parking Lots of the Fernando May Campus at Universidad del Bío-Bío" is submitted to fulfill the requirements demanded by the Universidad del Bío-Bío for obtaining the university degree in Civil Engineering in Computer Science.

Parking management has become an increasing problem as urban areas continue to grow, and more vehicles are on the roads. Consequently, the available parking spaces do not satisfy the demand. As a result, drivers spend a significant amount of time searching for available spots. This leads to several issues, including stress, pollution, time loss, and fuel wastage.

The present research aims to evaluate and contrast different types of models belonging to the YOLO convolutional neural network architecture. The primary objective is to identify potential improvements in the current management of the Universidad del Bío-Bío's parking facilities using artificial intelligence. For this purpose, a comparison between various variants of the YOLOv5 and YOLOv8 models was conducted. Each model was trained using two distinct datasets. The first dataset, named "VehículosUBB," focuses specifically on vehicle detection, while the second, "EspaciosUBB," aims to identify the status of parking spaces, whether available or occupied.

The results obtained reveal a significant disparity between the models trained with the "EspaciosUBB" dataset compared to those trained with the "VehículosUBB" dataset. The metrics used reflect a substantially superior performance in detecting parking spaces as opposed to vehicle detection. This finding suggests greater effectiveness and precision in the predictive capability of the models when focused on identifying available or occupied spaces.

Additionally, this study presents the creation of a mobile software aimed at highlighting the practical application of the model's predictions. This mobile software has been designed with a robust structure that enables users to interact effectively with the predicted data. This practical approach not only highlights the utility of the models in the specific context of the Universidad del Bío-Bío but also sets a valuable precedent for implementing similar solutions in other settings.

The relevance of these results transcends mere technical efficiency, as they have the potential to positively impact management and quality of life within the university campus. The ability to optimize the use of parking spaces can have tangible benefits in terms of traffic flow, logistical efficiency, and convenience for the university community, thus contributing to a substantial improvement in the daily campus experience.

Agradecimientos y Dedicatorias

Nos gustaría expresar nuestra más profunda gratitud y reconocimiento a todas aquellas personas que nos han apoyado y guiado a lo largo de este camino universitario, el cual ha estado lleno de desafíos y aprendizaje, definiéndonos como las personas que somos actualmente.

En primer lugar, queremos agradecer a nuestros profesores, cuya sabiduría, paciencia y dedicación han sido fundamentales en nuestro desarrollo académico y personal. Sus enseñanzas no solo nos han proporcionado el conocimiento profesional necesario, sino que también han fomentado en nosotros un espíritu crítico, y una pasión por el aprendizaje que perdurará más allá de nuestra vida universitaria. Su guía constante y consejos alentadores han sido un faro de luz en momentos de incertidumbre y desafío.

Pero en especial queremos agradecer a nuestros padres, a quienes les extendemos nuestro más sincero agradecimiento. Han sido nuestra fuente de amor, apoyo y fortaleza incondicional. Su fe inquebrantable en nuestras capacidades y su sacrificio constante han sido la base que nos ha permitido cumplir nuestros anhelados sueños. Su ejemplo de perseverancia y dedicación es algo que llevaremos siempre con nosotros y que nos ha enseñado el verdadero valor del esfuerzo y la constancia.

Además, agradecer a nuestros compañeros y amigos de carrera, quienes han compartido con nosotros esta aventura, brindándonos su apoyo y amistad. Han sido una parte esencial de esta experiencia universitaria, y los recuerdos compartidos serán atesorados por siempre.

Y finalmente, extendemos nuestro agradecimiento a todos aquellos que, de una forma u otra, han contribuido en nuestra vida para llegar a este momento. Cada palabra de ánimo, cada gesto de apoyo ha sido un valioso aporte en este camino.

Agradecidos de estos hechos:

Rodrigo Ignacio Carrasco Cofré y José Patricio Rivera Medina

Índice de contenido

1.	Introducción	13
1.1	Justificación	13
1.2	Propósito	14
1.3	Resumen de capítulos	15
2.	Problemática	16
2.1	Objetivo General	17
2.2	Objetivos Específicos.....	17
2.3	Hipótesis.....	17
3.	Estado del Arte	18
3.1	Preguntas de investigación	18
3.2	Palabras Clave	19
3.3	Criterios	19
3.3.1	Inclusión	19
3.3.2	Exclusión.....	19
3.4	Búsqueda y resultados	20
3.5	Conclusiones.....	24
4.	Marco Teórico	25
4.1	Inteligencia artificial	25
4.2	Machine Learning.....	25
4.3	Deep Learning	25
4.4	Visión artificial por computadora.....	26
4.5	Redes Neuronales Artificiales.....	26
4.6	Redes Neuronales Convolucionales	27
4.6.1	Capa de Convolución	28
4.6.2	Capa de Activación	28
4.6.3	Capa de Reducción (Pooling).....	32
4.6.4	Capa Totalmente Conectada (Fully Connected).....	32
4.7	YOLO (You Only Look Once)	33
5.	Métricas de evaluación	34
5.1	Precision	35
5.2	Recall	35

5.3	F1-Score.....	35
5.4	IoU	36
5.5	mAP	36
6.	Herramientas.....	37
6.1	Lenguajes de Programación	37
6.1.1	Python	37
6.1.2	Dart.....	37
6.2	Bibliotecas	38
6.2.1	OpenCV.....	38
6.2.2	Pytorch	38
6.2.3	Matplotlib.....	38
6.2.4	Ultralytics	39
6.2.5	NumPy	39
6.2.6	pandas	39
6.3	Base de Datos.....	40
6.3.1	MySQL	40
6.4	Etiquetado	40
6.4.1	Labellmg	40
6.5	Entornos de Desarrollo.....	41
6.5.1	Visual Studio Code.....	41
6.5.2	Google Colaboraty.....	41
6.5.3	Flutter	41
6.6	Hardware.....	42
6.6.1	Cámara	42
7.	Metodología de Desarrollo	43
7.1	Aspectos Generales.....	43
7.2	Etapa 1 y 2 “Entendimiento del negocio y los datos”	43
7.3	Etapa 3 “Obtención y preparación de los datos”	44
7.4	Etapa 4 Modelado	50
7.5	Evaluación	57
7.5.1	Caso 1: Resultados de Dataset “EspaciosUBB”	57
7.6	Implementación	79
8.	Conclusiones.....	85

8.1	Limitaciones	86
9.	Trabajos Futuros.....	87
10.	Referencias.....	88
11.	Anexo.....	91

Índice de tablas

Tabla 1: Resultados de búsqueda primaria en bases de datos	20
Tabla 2: Artículo seleccionado N°1.....	21
Tabla 3: Artículo seleccionado N°2.....	21
Tabla 4: Artículo seleccionado N°3.....	22
Tabla 5: Artículo seleccionado N°4.....	22
Tabla 6: Artículo seleccionado N°5.....	23
Tabla 7: Artículo seleccionado N°6.....	23
Tabla 8: Comparativa entre los modelos de YOLO para el dataset EspaciosUBB	67
Tabla 9: Comparativa de los modelos de YOLO para el dataset VehículosUBB.....	77

Índice de figuras

Figura 25: Caro Gutiérrez. (2005). Representación de la adaptación de la metodología Kitchenham https://www.researchgate.net/publication/251671565_Analisis_y_revision_de_la_literatura_en_el_contexto_de_proyectos_de_fin_de_carrera_Una_propuesta	18
Figura 1: Diferencias entre IA, Machine Learning y Deep Learning	26
Figura 2: Funcionamiento de visión artificial	26
Figura 3: Estructura de Redes Neuronales Artificiales	27
Figura 4: Extracción de características en la convolución	28
Figura 5: Gráfico de la función ReLu	29
Figura 6: Gráfico de la función Sigmoid	30
Figura 7: Gráfico de la función Leaky ReLu	31
Figura 8: Matriz de reducción de características	32
Figura 9: Autor desconocido. (n.d). Funcionamiento de arquitectura YOLO Aprende Machine Learning https://www.aprendemachinelearning.com/modelos-de-deteccion-de-objetos/	33
Figura 10: Funcionamiento de matriz de confusión	34
Figura 11: Logotipo de Python	37
Figura 12: Logotipo de Dart.....	37
Figura 13: Logotipo de OpenCV	38
Figura 14: Logotipo de Pytorch	38
Figura 15: Logotipo de Matplotlib.....	38
Figura 16: Logotipo de Ultralytics	39
Figura 17: Logotipo de NumPy	39
Figura 18: Logotipo de Pandas	39
Figura 19: Logotipo de MySQL	40
Figura 20: Logotipo de LabelImg	40
Figura 21: Logotipo de Visual Studio Code.....	41
Figura 22: Logotipo de Google Colab	41
Figura 23: Logotipo de Flutter	41
Figura 24: Cámara de seguridad Hikivision	42
Figura 26: Autor desconocido (s.f). Esquema del ciclo CRISP-DM estándar https://rpubs.com/mavaleriacc/1075094	43
Figura 27: Clases del dataset COCO	45
Figura 28: Posición y ángulo de la cámara al estacionamiento	46
Figura 29: Selección de primera imagen perfecta y eliminación de segunda imagen por ruido.....	46
Figura 30: Similitud de imágenes, con selección de la primera y eliminación de la segunda.....	47
Figura 31: Clase para el modelo de detección de vehículos	47
Figura 32: Clases para el modelo de detección de espacios	48
Figura 33: Etiquetado de imágenes para el dataset VehiculosUBB	48
Figura 34: Etiquetado para el dataset de EspaciosUBB	49
Figura 35: Ruta y clases para la importación de YOLO.....	51
Figura 36: Ultralytics. (n.d). Resultados promedios de las versiones de arquitecturas de YOLOv5 https://docs.ultralytics.com/models/yolov5/#key-features	51
Figura 37: Ultralytics. (n.d). Resultados promedios de las versiones de arquitecturas de YOLOv8 https://docs.ultralytics.com/models/yolov5/#key-features	52
Figura 38: Ultralytics. (n.d). Comparativa general de precisión entre las versiones de YOLO	

https://docs.ultralytics.com/models/yolov8/	52
Figura 39: Entrenamiento de un modelo de 100 épocas	54
Figura 40: Ajuste de hiperparámetros	54
Figura 41: Cantidad de capas de las versiones de YOLOv8	56
Figura 42: Cantidad de capas de las versiones de YOLOv5	56
Figura 43: Comparación de mAP de versiones de YOLOv5 con dataset de Espacios.....	58
Figura 44: Métricas de validación para entrenamiento de versiones de YOLOv5 con dataset de Espacios.....	59
Figura 45: Matriz de confusión normalizada de YOLOv5m.....	59
Figura 46: Matriz de confusión normalizada de YOLOv5n	59
Figura 47: Matriz de confusión normalizada de YOLOv5s	60
Figura 48: Métricas de F1-Score y Recall de YOLOv5 con dataset "EspaciosUBB"	61
Figura 49: Comparación de mAP de versiones de YOLOv8 con dataset de Espacios.....	62
Figura 50: Métricas de validación para entrenamiento de versiones de YOLOv8 con dataset de Espacios.....	63
Figura 51: Matriz de normalización de YOLOv8s	63
Figura 52: Matriz de normalización de YOLOv8m.....	63
Figura 53: Matriz de normalización de YOLOv8n.....	64
Figura 54: Métricas de F1-Score y Recall de YOLOv8 con dataset "EspaciosUBB"	65
Figura 55: Gráfico comparativo entre YOLOv5m y YOLOv8m.....	66
Figura 56: Comparativa entre métricas de evaluación entre YOLOv5m y YOLOv8m	67
Figura 57: Comparación de mAP de versiones de YOLOv5 con dataset de Vehículos.....	68
Figura 58: Métricas de validación para entrenamiento de versiones de YOLOv5 con dataset de Vehículos	69
Figura 59: Matriz de confusión normalizada de YOLOv5m.....	69
Figura 60: Matriz de confusión normalizada de YOLOv5m.....	69
Figura 61: Matriz de confusión normalizada de YOLOv5n	70
Figura 62: Métricas de F1-Score y Recall de YOLOv5 con dataset "VehículosUBB"	71
Figura 63: Comparación de mAP de versiones de YOLOv8 con dataset de Vehículos.....	72
Figura 64: Métricas de validación para entrenamiento de versiones de YOLOv8 con dataset de Vehículos	73
Figura 65: Matriz de Normalización de YOLOv8m	73
Figura 66: Matriz de Normalización de YOLOv8s.....	73
Figura 67: Matriz de Normalización de YOLOv8n	74
Figura 68: Métricas de F1-Score y Recall de YOLOv8 con dataset "VehículosUBB"	75
Figura 69: Comparativa del mAP entre YOLOv5s y YOLOv8m	76
Figura 70: Comparativa de métricas de evaluación entre los modelos YOLOv5s y YOLOv8m	77
Figura 71: Esquema del funcionamiento del proyecto	79
Figura 72: Coordenadas detectadas por el modelo	80
Figura 73: Coordenadas de ROI en el estacionamiento	80
Figura 74: Base de datos en MySQL del estacionamiento	81
Figura 75: Creación de endpoints para manipular los espacios.....	81
Figura 76: Registro Base de Datos.....	82
Figura 77: Interfaz de la aplicación para detectar estacionamientos	83
Figura 78: Comparación de la detección de la cámara con la aplicación.....	84

1. Introducción

Es de conocimiento general que los conductores que transitan por las ciudades enfrentan diversos desafíos, siendo uno de los más significativos la búsqueda de estacionamiento. Esta situación se intensifica en sitios con alta afluencia de personas, como colegios, universidades, hospitales, centros comerciales y supermercados. En la gran mayoría de estos lugares, la gestión del estacionamiento se realiza de manera manual e independiente, lo que obliga a los conductores a buscar un espacio disponible por su cuenta, sin saber dónde estacionarse directamente. Este escenario provoca una serie de inconvenientes, destacando la pérdida de tiempo y el aumento del estrés.

Este proyecto se centra en mejorar la gestión de estacionamiento en las dependencias de la Universidad del Bío-Bío, específicamente en el campus Fernando May, sede Chillán. La problemática va más allá de la falta de organización de estacionamientos, ya que también implica la ausencia de sistemas que faciliten a los conductores la localización de un lugar disponible para estacionar de forma cómoda. La mayoría de los sitios que poseen aparcamientos carecen de herramientas tecnológicas para monitorear y controlar el flujo vehicular de manera efectiva. Por lo tanto, este proyecto busca implementar soluciones que no solo mejoren la experiencia de estacionamiento en el campus, sino que también contribuyan a la disminución del estrés y el uso eficiente del tiempo por parte de la comunidad universitaria.

1.1 Justificación

El parque automotriz de Chile ha presentado un constante crecimiento, donde cada año se elevan las cifras de personas que tienen acceso a un vehículo. *Usach (s.f.)* indica que desde el año 2010, la cantidad de estos medios de transporte ha crecido más del doble, con un 57%, lo cual consecuentemente, provoca una mayor demanda de espacios para estacionar.

En el aspecto psicológico, diversos estudios afirman que, buscar aparcamiento en estacionamientos puede generar considerables índices de estrés e incomodidad. Según un estudio publicado por la compañía Bosh, cerca del 52% de los españoles sufre de estrés y ansiedad al perder tiempo buscando estacionamientos (*Autocasión, 2013*). Así mismo la aplicación EasyPark, destaca que uno de cada cinco conductores se estresa a la hora de buscar aparcamiento (*Zamoranews, s.f-a*). Y finalmente, la página PsicoCode estima que la irritabilidad, agresividad e ira, además de los niveles de cortisol que generan el estrés, aumentan en situaciones donde conductores se enfrentan a buscar aparcamiento (*PsicoCode, s.f.*).

En el aspecto económico diversos estudios afirman que se puede llegar a malgastar grandes cantidades de combustible con este rudimentario sistema mencionado, ya que los vehículos al estar esperando y conduciendo lentamente en una demorosa búsqueda de lugar, llegan a consumir considerables cantidades de estos fluidos necesarios para alimentarlos. Según

un estudio elaborado por la aplicación WazyPark se estima que la población española tiene un gasto anual de 3.900 millones de euros al año en combustible (*Sara, 2020*).

En el aspecto ambiental, se destaca la existencia de estudios que explican que los vehículos en búsqueda de estacionamientos pueden llegar a producir considerables cifras de compuestos nocivos que impactan en el medioambiente. Según la revista digital MoviliBlog, sólo en la ciudad de Chicago, producto de la búsqueda de estacionamiento, se estima una producción de 129.000 toneladas de CO₂ al año (*Viñuela, 2015*).

Por último, se debe destacar que la ciudad de Chillán, donde se encuentra la Universidad bajo estudio, se ha convertido en capital regional de la región de Ñuble, creada en el año 2017, por lo cual últimamente, se ha notado un aumento en la densidad poblacional, así como en el uso de automóviles. Análisis estadísticos muestran un aumento del 23% en el flujo vehicular. Otro acontecimiento importante para destacar es el mayor acceso a vehículos, producto de mayor capacidad adquisitiva por parte de las personas, gracias a hechos como los masivos retiros de fondos de previsión social, entre otros (*SoyChile, 2023*).

1.2 Propósito

Se llevó a cabo una investigación aplicada con el fin de dar solución a la problemática anteriormente planteada. En principio, se estudiaron variadas formas de resolver esta necesidad, donde se obtuvo que uno de los métodos más instaurados, es el uso de sensores que detectan vehículos instalados en los espacios de estacionamiento, los cuales resultan ser sumamente costosos y poco prácticos, puesto que cada lugar debe poseer su respectivo sensor. Por lo tanto, la mejor solución es aprovechar el potencial de la inteligencia artificial por una serie de razones, entre, las que destacan, el aspecto económico mencionado anteriormente respecto a los sensores, la confiabilidad y precisión que presentan los modelos inteligentes y la escalabilidad al poder detectar una gran variedad de objetos (personas, patentes, rostros, etc.) en tiempo real. Se plantea como solución la selección de un modelo de visión artificial, que logre detectar los espacios de estacionamiento y clasificarlos según su estado (vacante u ocupado), esperando índices que superen el 95% de precisión, ya que generalmente se considera desde el 90% como un modelo que entrega información confiable, lo que aumenta así la satisfacción del usuario y la utilidad práctica del modelo.

Lo anterior se pudo lograr mediante la captura de imágenes en tiempo real, obtenidas de una cámara ubicada en un sitio estratégico, desde el cual se pueda tener una visibilidad de calidad del estacionamiento en cuestión.

1.3 Resumen de capítulos

El desarrollo del informe se divide en diez capítulos, siendo el primero el de introducción abordado anteriormente, donde se expone el problema de manera general. El segundo capítulo describe el problema a resolver de manera detallada, además de definir el objetivo general, los objetivos específicos y la hipótesis. En el capítulo tres se presenta el marco teórico donde se definen los conceptos relevantes a considerar para comprender el proyecto, como técnicas y arquitecturas de programación que se usarán posteriormente. En el cuarto capítulo se definen las métricas de evaluación y conceptos relacionados para comprender los resultados. Ya en el quinto capítulo se nombran las herramientas que se ocuparán para desarrollar el proyecto, como lenguajes de programación, bibliotecas, entornos de desarrollo, etc. En el sexto capítulo se presenta la revisión sistemática de la literatura realizada en base al tema de la investigación, donde, se estudia el estado actual de la literatura en casos similares al abordado. En el séptimo capítulo se presenta la metodología de desarrollo, la cual muestra la obtención y procesamiento de los datos, siguiendo con el modelado de las arquitecturas y sus respectivas funcionalidades. Luego se presenta la etapa de evaluación, donde se pone a prueba mediante métricas el rendimiento de los modelos, y se selecciona el que demuestre mejor rendimiento. El séptimo capítulo consta de la implementación del modelo en una aplicación que logre entregar al usuario la solución que el modelo plantea de forma comprensible y cómoda. El capítulo número ocho presenta las conclusiones obtenidas de la investigación. Finalmente, el capítulo número nueve indica los trabajos futuros a realizar donde se plantea ideas para escalar el proyecto.

2. Problemática

Este proyecto busca mejorar la gestión de estacionamiento en las dependencias de la Universidad del Bío-Bío en la sede Chillán, campus Fernando May. Actualmente, esta sede de la Universidad cuenta con un aproximado de 4200 estudiantes y 184 funcionarios (*Hermosilla, 2023*), los cuales son potenciales usuarios de los estacionamientos diariamente. Además, es necesario mencionar el hecho del aumento de la oferta académica que registra la Universidad actualmente, con la recién implementada carrera de Química y Farmacia en 2023 y la apertura de la carrera de Medicina en 2024. Con ello se espera una considerable repercusión en la cantidad de personas que necesiten ocupar estacionamientos.

La universidad cuenta con 7 sectores de estacionamiento en el campus Fernando May, que, en promedio, suman alrededor de 300 plazas de estacionamiento disponibles. Donde algunos de estos sectores suelen estar prácticamente repletos en las horas peak.

Actualmente, no existe un sistema que gestione de alguna forma la disposición del estacionamiento, por lo cual se plantea desarrollar una comparativa de modelos basados en inteligencia artificial, que, mediante el uso de cámaras que capturen datos visuales, los cuales serán sometidos a procesos y técnicas buscando mejorar su calidad, finalmente logren determinar la cantidad de espacios libres en los estacionamientos seleccionados, dando información al usuario para que pueda tomar decisiones.

Se opta por el uso de inteligencia artificial, puesto que esta propone imitar el comportamiento humano de manera que se puedan mejorar procesos cotidianos, supliendo necesidades presentes en el diario vivir. Además, hay que destacar que el uso de cámaras es un método eficiente y factible, superando incluso el ya conocido método con sensores.

Este estudio se centrará en un estacionamiento específico, ubicado en las dependencias del Campus Fernando May de la Universidad, el cual se encuentra al costado del Edificio de Laboratorios Centrales. La razón que justifica la selección de dicho sector de estacionamiento es su ubicación, que se halla a la entrada, pero no está a la vista de forma inmediata y además presenta un tamaño ideal, donde se dispone de 39 plazas.

Hay que destacar que este estudio, se presenta como un precursor para la adaptación en otros estacionamientos de la Universidad. La cual tiene en marcha planes de seguridad y televigilancia, por lo que a futuro se efectuará la instalación de una gran cantidad de cámaras a lo largo del campus y las zonas de estacionamiento, las cuales pueden ser aprovechadas para estas tareas de gestión vehicular en los estacionamientos mediante modelos de inteligencia artificial.

2.1 Objetivo General

Investigar y comparar modelos de inteligencia artificial basados en Deep Learning, con el propósito de utilizar cámaras para mejorar la eficiencia del estacionamiento en la Universidad del Bío-Bío.

2.2 Objetivos Específicos

A continuación, se establecen los objetivos específicos que permitirán enfocarse en alcanzar el objetivo general.

- Obtener imágenes del estacionamiento, etiquetarlas y crear dos conjuntos de datos con diferentes enfoques y clases para la detección.
- Investigar, comparar y aplicar modelos de Deep Learning, junto a un conjunto de datos que permita detectar los estados (libre u ocupado) de disponibilidad en el estacionamiento de la universidad mediante cámaras en tiempo real.
- Evaluar la eficiencia de los modelos a utilizar para detectar y registrar los espacios libres en el estacionamiento de la universidad.
- Diseñar una aplicación sencilla para que los usuarios tengan la capacidad de consultar sobre la disponibilidad de espacios.
- Integrar el modelo seleccionado con la aplicación para que los usuarios puedan consultar la disponibilidad de espacios de estacionamiento en tiempo real.

2.3 Hipótesis de Trabajo

La aplicación de un modelo basado en inteligencia artificial, a través del empleo de cámaras para la detección de espacios disponibles en estacionamientos dentro de la Universidad del Bío-Bío, permitirá agilizar el proceso de encontrar aparcamiento mediante una aplicación móvil, resultando en la reducción significativa de sus tiempos de búsqueda.

3.2 Palabras Clave

Se presentan los términos clave para realizar la búsqueda y selección de artículos, además de sus posibles combinaciones.

1. "Deep Learning (AND) Cameras"
2. "Deep Learning (AND) Car detection"
3. "Deep Learning (AND) Car detection (AND) Parking"
4. "Deep Learning (AND) Cameras (AND) Car detection"
5. "Deep Learning (AND) Cameras (AND) Car detection (AND) CNN"
6. "Deep Learning (AND) Cameras (AND) Car detection (AND) CNN (AND) Parking"

3.3 Criterios

Para la selección de fuentes de información, se definieron ciertos estándares o criterios que considerar al revisar en revistas, artículos, páginas web, libros, entre otros. Donde se espera rescatar información sobre el Deep Learning, redes neuronales, técnicas, e implementaciones realizadas.

3.3.1 Inclusión

- Los artículos deben estar directamente relacionados con Redes Neuronales Convolucionales (CNN) o sus aplicaciones
- Los artículos deben ser recientes y actualizados, solo tomando en cuenta aquellos publicados desde el año 2016
- Se prefieren aquellos artículos académicos revisados por pares, como revistas y actas de conferencias

3.3.2 Exclusión

- Los artículos que no tengan relación directa con las Redes Neuronales Convolucionales o que no sean aporte al tema serán excluidos
- Los artículos obsoletos o desactualizados serán excluidos por no contribuir al estado actual de la investigación
- Los artículos en idiomas diferentes al inglés o español serán excluidos

3.4 Búsqueda y resultados

En esta etapa se realiza la búsqueda de artículos en las bases de datos a las que se tiene acceso, en donde se definen las diferentes combinaciones con los términos claves y se obtienen los siguientes resultados de búsqueda primaria como muestra la Tabla 1, donde queda seleccionar artículos basándose en criterios de inclusión y exclusión.

Términos/Base de Datos	Dialnet	Web of Science	IEEEExplore	Resultados
(Deep Learning) AND (Cameras)	48	5564	7434	13046
(Deep Learning) AND (Car Detection)	20	499	932	1451
(Deep Learning) AND (Car Detection) AND (Cameras)	6	106	268	380
(Deep Learning) AND (Car Detection) AND (Cameras) AND (CNN)	2	35	70	107

Tabla 1: Resultados de búsqueda primaria en bases de datos

De los resultados de la búsqueda primaria realizada anteriormente, se seleccionaron un total de 35 artículos relacionados al problema en específico, de los cuales finalmente se utilizaron que se consideran relevantes en sus aportes a la investigación. Estos estudios fueron escogidos bajo los siguientes criterios de selección:

- Se incluyen aquellos artículos que tienen relación con el contexto del problema planteado.
- Se incluyen aquellos artículos que muestran métodos efectivos y funcionales.
- Se excluyen aquellos artículos duplicados, o que expliquen la misma idea.

A continuación, se presentan los 6 artículos, de diferentes autores y con variadas perspectivas basadas en la problemática de la investigación. Se describe de manera resumida la relevancia de cada artículo y su utilidad para esta investigación.

Artículo N°1 (Rahman, 2020).	
Nombre	Convolutional Neural Network Customization for Parking Occupancy Detection.
Revista/Conferencia	International Conference on Engineering and Informatics (ICELTICs) 2020.
Fecha de publicación	28 de octubre de 2020.
Autores	Sayuri Rahman, Marwan Ramli, Fitri Arnia, Arnes Sembiring, Rusdha Muharar.
Aporte al estudio	Comparativa de 5 modelos CNN en diferentes escenarios, donde mAlexNet, y CmAlexNet (Arquitectura creada por los autores), logran los mejores resultados en eficacia y eficiencia.
Observación	Se obtiene que CmAlexNet lidera la eficacia con un 99.12% y mAlexNet le sigue con un 99.09% y en cuanto a tiempo en entrenamientos mAlexNet es superior por un 1.25%. Se utiliza el conjunto de datos PKLot. Se hace uso de técnicas de data Aumentation.

Tabla 2: Artículo seleccionado N°1

Artículo N°2 (Zhang, 2019).	
Nombre	Smart Parking System Based on Convolutional Neural Network Models.
Revista/Conferencia	6th International Conference on Information Science and Control Engineering (ICISCE) 2019.
Fecha de publicación	20 de diciembre de 2019.
Autores	Wenjin Zhang, Jason Yan, Cui Yu.
Aporte al estudio	Creación de 2 modelos CNN, VGGExtended y MiniParkCNN las cuales muestran buenos resultados, recíprocos en eficacia y eficiencia respectivamente. Uso del algoritmo Canny y proceso de segmentación y minimización de ruido en imágenes.
Observación	La variación en tiempo y eficacia de las arquitecturas radica en la cantidad de capas y complejidad de estas. VGG extended muestra 99.4 % de eficacia en detección y MiniParkCNN muestra 99.3%. Y en relación con el tiempo VGG se tarda una razón de 7:1 segundos más que MiniParkingCNN en las pruebas.

Tabla 3: Artículo seleccionado N°2

Artículo N°3 (Duy-Linh Nguyen, 2023)	
Nombre	YOLO5PKLot: A Parking Lot Detection Network Based on Improved YOLOv5 for Smart Parking Management System
Revista/Conferencia	Communications in Computer and Information Science, vol 1857
Fecha de publicación	1 de agosto de 2023
Autores	Duy-Linh Nguyen, Xuan-ThuyVo, Adri Priadana, Kang-Hyun Jo
Aporte al estudio	Uso de la arquitectura Yolov5 en sus diferentes versiones, además del uso de técnicas como BottleNeck y SpatialPyramidPooling. Se hace uso del dataset PKLot para el entrenamiento.
Observación	Se obtiene una precisión exitosa con un promedio del 99,6%. Y esto utilizando una complejidad computacional menor a la del modelo Yolov5 original. Se mencionan limitaciones respecto al clima adverso.

Tabla 4: Artículo seleccionado N°3

Artículo N°4 (Weiling Chen, 2019).	
Nombre	Unauthorized Parking Detection using Deep Networks at Real Time
Revista/Conferencia	2019 IEEE International Conference on Smart Computing (SMARTCOMP)
Fecha de publicación	12 de junio de 2019
Autores	Weiling Chen y Chai Kiat Yeo
Aporte al estudio	Uso de la arquitectura Yolov3 para detección de vehículos, usando el Dataset COCO, y recurriendo a <u>imágenes</u> capturadas propias y haciendo uso de Data Aumentation.
Observación	Se obtiene una precisión exitosa del 95.1%, un Accuaracy del 96.3% y un Recall del 90.2%, resultando en un modelo exitoso.

Tabla 5: Artículo seleccionado N°4

Artículo N°5 (Nguyen T., 2021).	
Nombre	An Adaptive Vision-based Outdoor Car Parking Lot Monitoring System.
Revista/Conferencia	2020 IEEE Eighth International Conference on Communications and Electronics (ICCE).
Fecha de publicación	13 de enero de 2021.
Autores	Thang Nguyen*, Thom Tran, Tho Mai, Hanh Le, Cuong Le, Doan Pham, Kieu-Ha Phung.
Aporte al nuestro estudio	Crean su propio conjunto de datos llamado HUSTPark, y se hace uso de la arquitectura malexNet, se utilizan técnicas como image-cropping para el procesamiento de imágenes, y técnicas como ROI.
Observación	Extendieron el conjunto de datos CNRPark agregando HUSTPark <u>para</u> entrenar su modelo (mAlexNet) obteniendo un 97% de precisión. Se utilizaron métricas como Recall, f1-score y AUC.

Tabla 6: Artículo seleccionado N°5

Artículo N°6 (Thomas T., 2018).	
Nombre	Smart Car Parking System Using Convolutional Neural Network.
Revista/Conferencia	2018 International Conference on Inventive Research in Computing Applications (ICIRCA)
Fecha	11 de julio de 2018.
Autores	Tom Thomas, Tarun Bhatt.
Aporte al nuestro estudio	Utilizan el conjunto de datos PKLot para entrenar el modelo escogido (mAlexNet), explican cómo funciona cada una de las capas del modelo.
Observación	Utilizan técnicas como Canny y ROI.

Tabla 7: Artículo seleccionado N°6

3.5 Conclusiones

De entre los diversos artículos examinados, se destacan dos arquitecturas distintas para la evaluación de los estados de los espacios de estacionamiento: mAlexNet, reconocida como la mejor en la clasificación de objetos, y YOLO, identificada como la más eficaz para la detección de dichos objetos. La elección se orienta hacia YOLO, dado su estatus como la arquitectura más contemporánea, su capacidad para detectar una mayor variedad de objetos, su menor saturación en estudios en comparación con mAlexNet, y su idoneidad para implementaciones en tiempo real.

Se valora la composición de las capas que integran la arquitectura YOLO, así como las técnicas empleadas, como la Región de Interés (ROI) y el método de DropOut. La comprensión de estos componentes resulta esencial para el desempeño óptimo de la arquitectura en la detección de espacios de estacionamiento. Además, se presta atención a las métricas necesarias para evaluar el rendimiento del modelo, facilitando así comparaciones rigurosas con otros enfoques.

No obstante, no se subestima la importancia de la obtención y preparación adecuada de un conjunto de datos para el entrenamiento de la arquitectura. Se reconoce que la calidad y diversidad del conjunto de datos influyen directamente en la capacidad del modelo para generalizar y realizar detecciones precisas en diversos contextos. Por lo tanto, se considera fundamental abordar con atención la fase de adquisición y preparación del dataset para garantizar resultados confiables y aplicables.

Finalmente, la selección de YOLO se fundamenta en criterios específicos que van más allá de su eficiencia en la detección de objetos, abarcando aspectos como su versatilidad, relevancia actual, y capacidad para operar en tiempo real. La profundización en la comprensión de sus componentes y el enfoque meticuloso en la preparación del dataset subrayan un enfoque integral para maximizar la efectividad de esta arquitectura en el contexto de la detección de espacios de estacionamiento.

4. Marco Teórico

4.1 Inteligencia artificial

En 1950 Alan Turing propuso la Prueba de Turing, la cual consiste en evaluar si una máquina puede demostrar un comportamiento inteligente equivalente al de un ser humano. La máquina superará la prueba si un evaluador humano, no puede diferenciar si la respuesta proviene de una computadora o de un humano. De esta manera se dio un concepto a “inteligencia”, estableciendo un punto de referencia para evaluar el progreso de la inteligencia artificial (IA).

Se define como una rama de la informática, que tiene como objetivo el desarrollo de sistemas que puedan simular el comportamiento humano, siendo estos capaces de percibir su entorno, permitiendo de esta manera razonar, aprender y comunicarse con el fin de tomar decisiones de manera autónoma.

Actualmente la inteligencia artificial es una pieza fundamental dentro de nuestra sociedad, siendo una tecnología revolucionaria que se puede utilizar en todos los campos (*Russel & Norvig, 2004*).

4.2 Machine Learning

El machine Learning es una rama de la inteligencia artificial que, mediante una serie de datos etiquetados, permite a un modelo descubrir patrones que le ayudaran en un futuro a predecir datos que no se encuentran etiquetados.

Para esto, los datos se preparan, se limpian y se organizan facilitando al modelo las características que debe aprender. El modelo se entrena para medir su porcentaje de aprendizaje con métricas de evaluación, si el modelo logra “aprender” no tendrá problemas al momento de clasificar datos que no haya visto antes, pero en caso de que los datos no se utilicen y se limpien de manera correcta, el modelo puede sufrir de sesgo y alejarse de la predicción real (*Datascientest, 2021*).

4.3 Deep Learning

El Deep Learning es una rama del machine Learning, que tiene como objetivo reproducir las acciones del cerebro humano, funcionando mediante capas interconectadas, denominadas “redes neuronales artificiales”, donde cada capa recibe y procesa con algoritmos la información de la capa anterior para realizar predicciones (*Datascientest, 2022*).

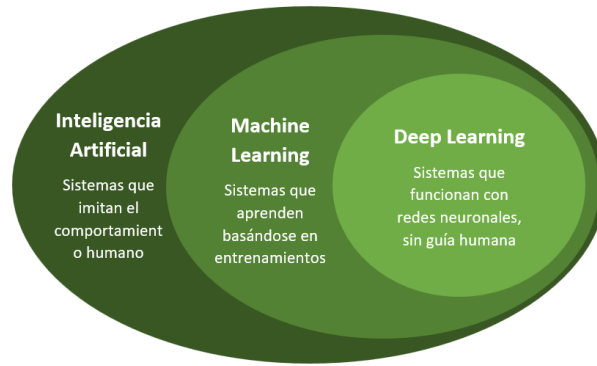


Figura 2: Diferencias entre IA, Machine Learning y Deep Learning

4.4 Visión artificial por computadora

La visión artificial, es una rama de la inteligencia artificial que consiste en entregar a la computadora la capacidad de percibir y comprender una imagen de manera similar a como la realiza un ser humano (IBM, s.f.). Como se muestra en la Figura 3, la visión por computadora es un proceso que consta de una serie de etapas que se aplican a la imagen antes de extraer sus características. En un comienzo, se preprocesa con el objetivo de mejorar la calidad de la imagen, luego se segmenta para analizar las regiones significativas de esta, finalmente se extraen las características como bordes, esquinas, texturas, etc. Las cuáles serán relevantes para su interpretación.

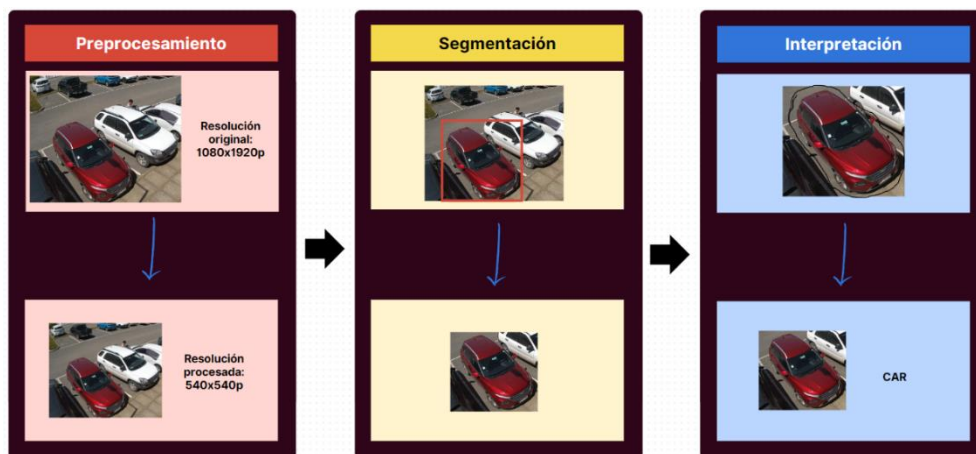


Figura 3: Funcionamiento de visión artificial

4.5 Redes Neuronales Artificiales

Una red neuronal artificial (RNA), es un modelo que tiene como objetivo el procesamiento de la información basado en el funcionamiento de las redes neuronales del cerebro humano. La red está compuesta por un conjunto de unidades de procesamiento llamadas neuronas, las cuales están conectadas a otras a través de pesos asociados que determinan la influencia de las conexiones entre estas neuronas.

La *Figura 4* muestra un esquema de una red neuronal artificial, esta se compone de una capa de entrada en la que cada nodo(neurona) representa una característica diferente, las capas ocultas procesan esta información para finalmente entregar una salida con la clasificación o predicción del modelo dependiendo el caso.

Mientras más capas contenga una red neuronal más compleja es y por lo tanto puede representar tareas de aprendizaje automático de manera más efectiva (*Fernandez & Florez, 2008*).

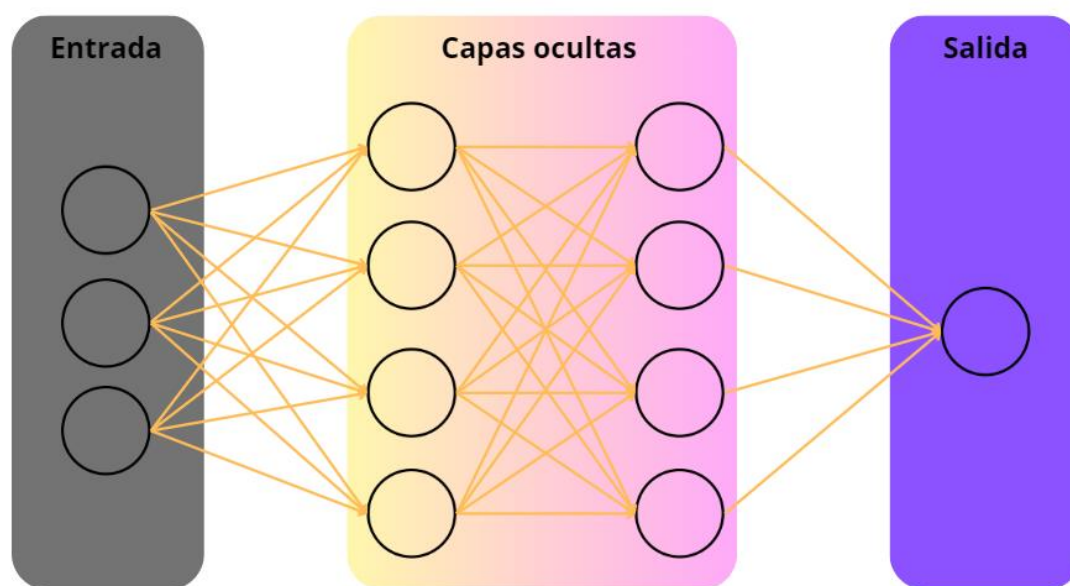


Figura 4: Estructura de Redes Neuronales Artificiales

4.6 Redes Neuronales Convolucionales

Una red neuronal convolucional, es un tipo de red neuronal diseñada específicamente para procesar datos que tengan estructuras cuadriculadas, como imágenes y videos. Siendo eficaces para realizar tareas de visión por computadora como la detección de objetos, clasificación y segmentación de imágenes. Lo especial de este tipo de redes es su capacidad para realizar operaciones de convolución, que mediante aplicaciones matemáticas y filtros se extraen características locales como bordes, texturas y patrones en las imágenes. La arquitectura de estas redes está compuesta por las capas que se explican a continuación (*IBM, s.f*).

4.6.1 Capa de Convolución

La capa de convolución es la encargada de aplicar filtros a través de matrices que recorren la imagen, este recorrido es lo que se conoce como “convolución” teniendo como objetivo la extracción de características (IBM, s.f).

El filtro contiene números que representan el patrón definido, los cuales se deslizan desde la esquina superior izquierda de la imagen hacia la derecha de esta con una cantidad específica de píxeles para realizar el producto punto entre el filtro y los datos de entrada, luego cuando llega al borde de la imagen continua con el proceso hacia abajo hasta cubrir la imagen de manera total obteniendo como resultado un mapa de características que entrega la presencia del patrón buscado como se puede ver en la *Figura 5*.

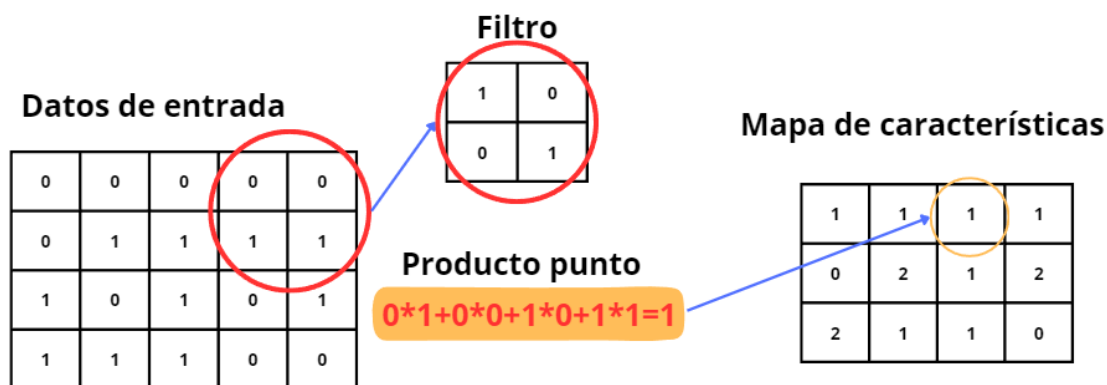


Figura 5: Extracción de características en la convolución

4.6.2 Capa de Activación

Es la capa encargada de introducir dentro de la red neuronal la no linealidad, con el fin de aprender y hacer predicciones más complejas, generalmente esta capa se suele utilizar luego de una capa de convolución o una de Max Pooling (IBM, s.f). Existen varias funciones de activación dentro de las más comunes se encuentran:

- **ReLU (Rectified Linear Activation)**

Como se puede observar en la función de la *Figura 6*, si el valor que recibe como entrada es positivo la función mantiene el valor, si es negativo la función le asigna 0, agregando de esta manera la no linealidad a la red (*IBM, s.f*).

$$f(x) = \max(0, x)$$

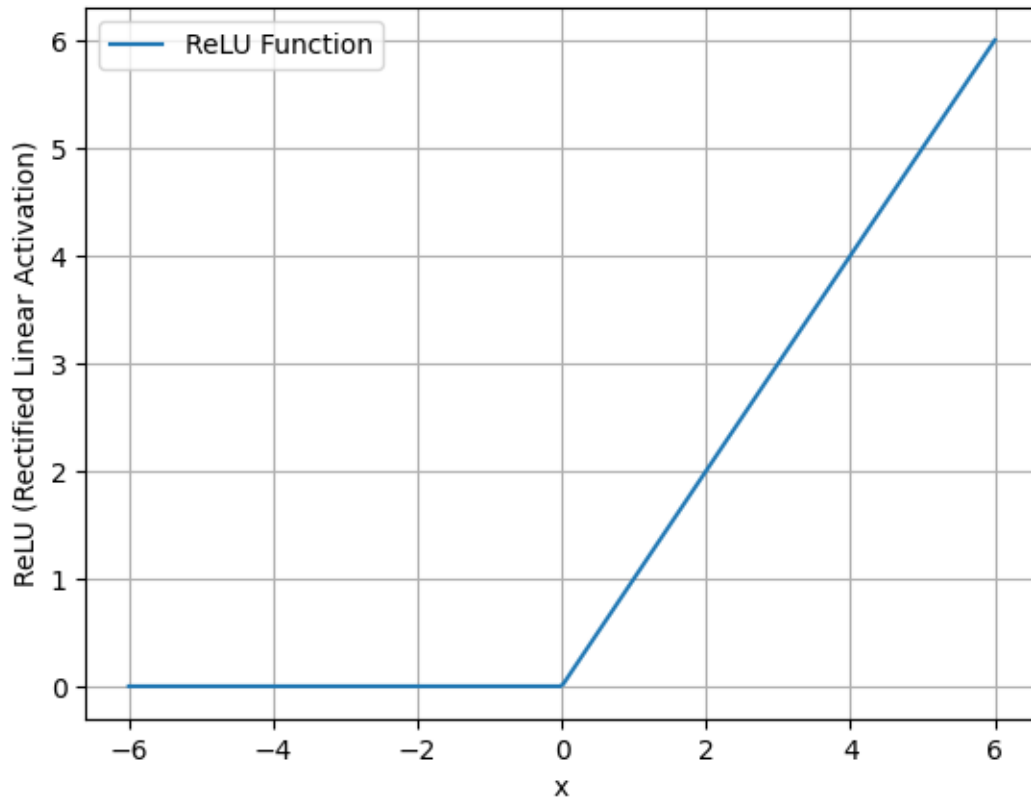


Figura 6: Gráfico de la función ReLu

- **Sigmoid**

La siguiente función que se aprecia en la *Figura 7* genera valores entre un rango de 0 y 1, siendo de este modo la más adecuada para problemas de clasificación binaria, el valor entregado por esta función es una probabilidad de pertenecer a una de las 2 clases de un modelo (*IBM, s.f*).

$$f(x) = \frac{1}{(1 + e^{-x})}$$

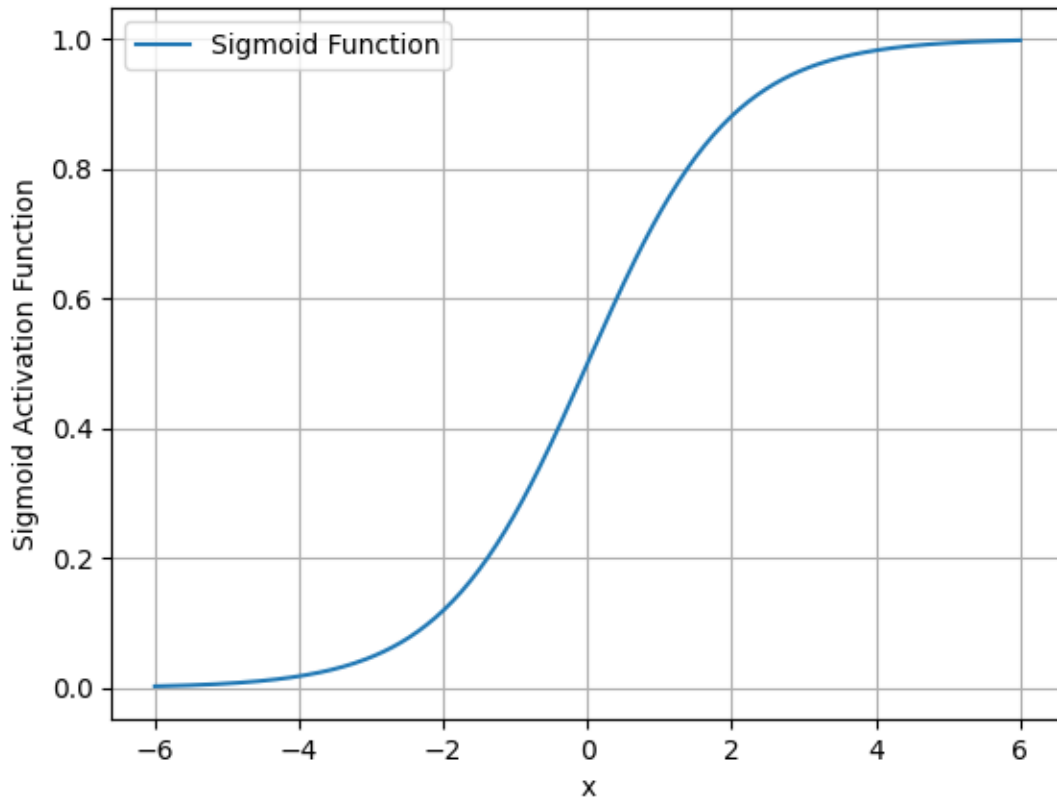


Figura 7: Gráfico de la función Sigmoid

- **Leaky ReLU**

La función representada en la

Figura 8 es una variación de ReLU, se diferencia en la posibilidad de permitir un valor pequeño para las entradas que sean menores o iguales a 0, con el objetivo de evitar que las neuronas se vuelven inactivas y dejen de aprender (*IBM, s.f.*).

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ a \cdot x & \text{for } x \geq 0 \end{cases}$$

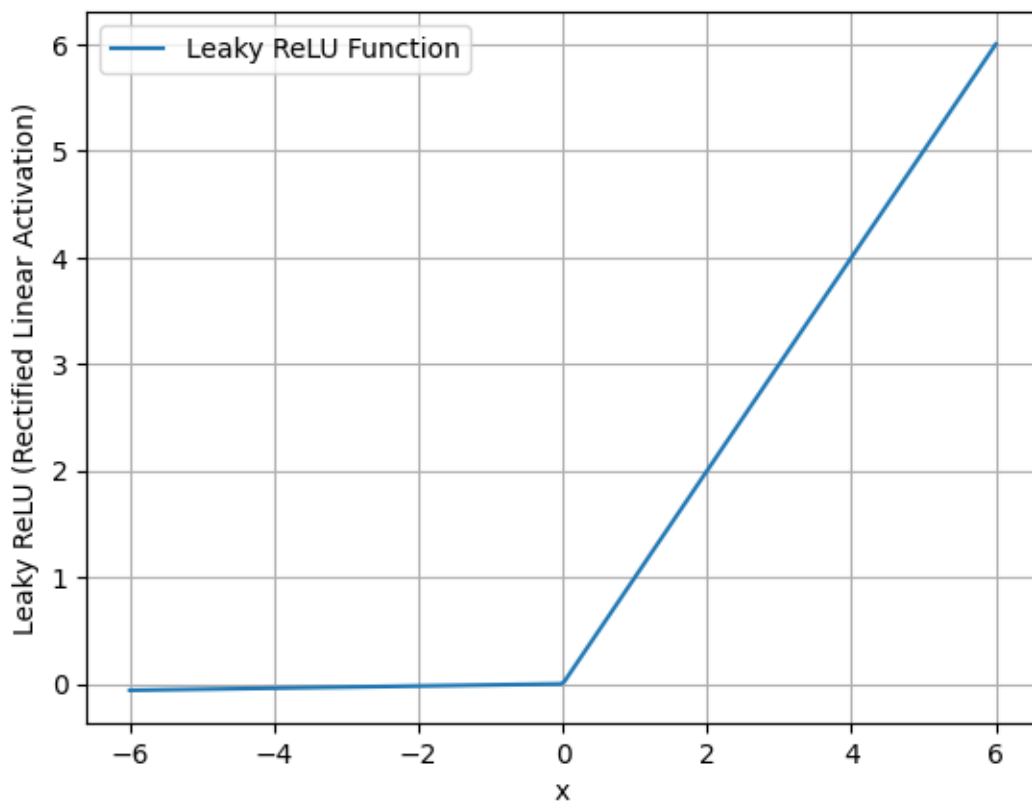


Figura 8: Gráfico de la función Leaky ReLU

4.6.3 Capa de Reducción (Pooling)

Esta capa se suele utilizar luego de una capa de convolución y tiene como finalidad reducir la dimensión espacial de las características obtenidas en el proceso anterior. Funciona mediante matrices que recorren la salida de la capa de convolución generalmente de tamaño 2x2 o 3x3, estas matrices sacan el valor máximo o el promedio de los datos obteniendo en consecuencia una representación dimensional más pequeña y robusta (IBM, s.f).

Se puede apreciar en la *Figura 9* una reducción dimensional de 2x2 sobre una matriz de características, quedando el valor máximo como salida en cada uno de los casos.

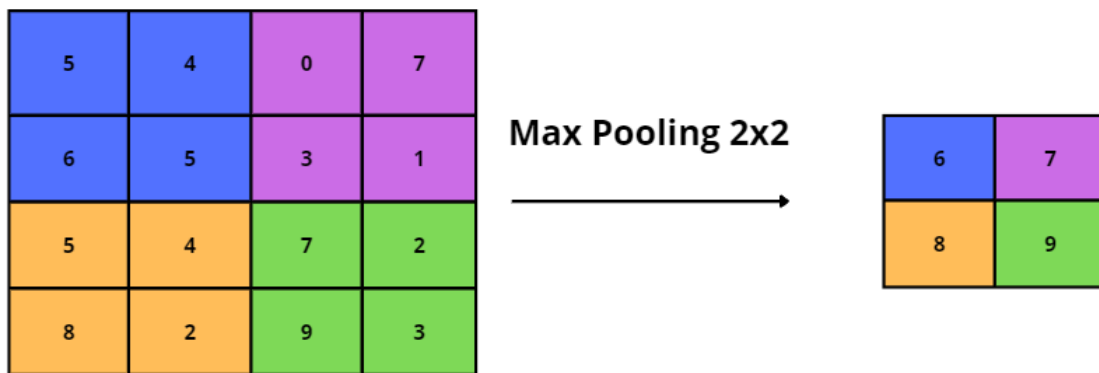


Figura 9: Matriz de reducción de características

4.6.4 Capa Totalmente Conectada (Fully Connected)

Esta capa como su nombre indica, conecta cada uno de los nodos de los procesos anteriormente descritos, con la finalidad de extraer y procesar información para arrojar una probabilidad de pertenecer a una clase del modelo.

El número de neuronas en esta capa indica el tamaño de la salida, es decir la cantidad de clases del modelo será la cantidad de neuronas que tendrá la capa totalmente conectada (IBM, s.f).

4.7 YOLO (You Only Look Once)

Es una arquitectura CNN, que tiene como propósito principal, detectar y encontrar objetos en imágenes, aunque de una manera más específica, se puede afirmar que es una red entrenada para predecir los límites de un objeto dentro de un cuadro y así definir la respectiva clase a la que pertenece, siendo efectivamente capaz de procesar imágenes en tiempo real.

YOLO funciona de forma que divide a la imagen en N cuadrículas, donde cada una tiene un respectivo sector dimensional $S \times S$ de igual tamaño. Y cada una de estas cuadrículas se encarga de detectar y localizar el objeto que contiene. Luego cada una de estas cuadrículas pronostica N posibles Bounding Boxes, que son las cajas delimitadoras usadas para marcar y rodear al objeto de interés, entregando además el nombre de la clase del elemento y la probabilidad del umbral de confianza de dicha detección, resultando en la obtención de $N \times S \times S$ posibles cajas delimitadoras como se muestra en la

Figura 10. Y finalmente se realiza la tarea de eliminar las cajas delimitadoras que presentan un bajo nivel de umbral de confianza, eliminando los cuadros de más y aplicando la técnica de supresión máxima para eliminar duplicados.

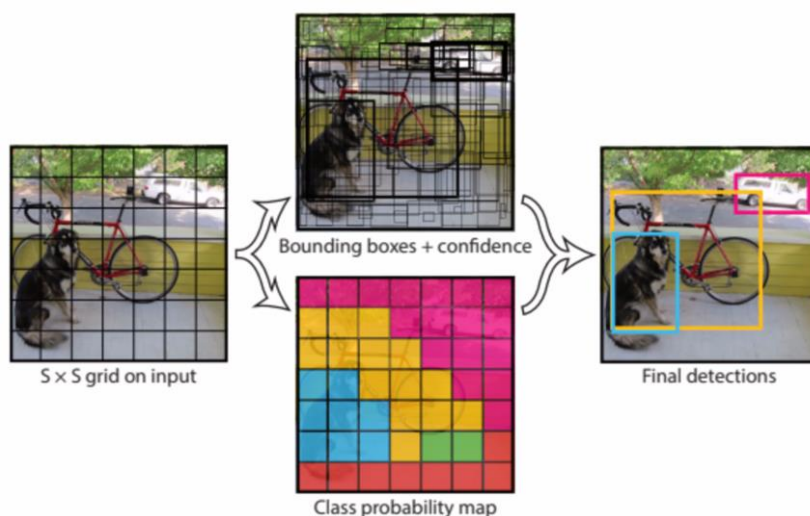


Figura 10: Autor desconocido. (n.d). Funcionamiento de arquitectura YOLO Aprende Machine Learning <https://www.aprendemachinelearning.com/modelos-de-deteccion-de-objetos/>

YOLO tiene una serie de características esenciales para la detección en tiempo real, dentro de las que destaca una alta velocidad de procesamiento realizando clasificaciones en un solo recorrido a la imagen, dando como resultado una correcta y limpia detección, además de presentar alto grado de efectividad y ser altamente flexible y escalable. Aunque también posee desventajas como baja efectividad en precisión de objetos pequeños, problemas de superposición y dificultades con objetos deformados (Jay, 2023).

5. Métricas de evaluación

Las métricas de evaluación son de utilidad para medir el rendimiento del modelo, y conocer cómo es su comportamiento en un entorno real. En principio, es necesario conocer algunos conceptos antes de definir las métricas, con tal de comprender como funcionan.

A modo de ejemplo para comprender de manera más adecuada, al evaluar el modelo, en el caso del estacionamiento se trabajó con dos clases, siendo “ocupado” la clase1 y “vacío” la clase2 en este caso, obteniendo los siguientes conceptos:

- Verdadero Positivo (VP): Se indica como la cantidad de casos totales que el modelo reconoce como objetos de la clase1 “ocupado”, cuando en realidad si son de la clase1.
- Verdadero Negativo (VN): Se indica como la cantidad de casos totales que el modelo reconoce como objetos de la clase2 “vacío”, cuando en realidad si son de la clase2.
- Falso Positivo (FP): Se indica como la cantidad de casos totales que el modelo reconoce como objetos de la clase1 “vacío”, cuando en realidad son objetos de la clase2 “ocupado”.
- Falso Negativo (FN): Se indica como la cantidad de casos totales que el modelo reconoce como objetos de la clase1 “ocupado”, cuando en realidad son objetos de la clase2 “vacío”.

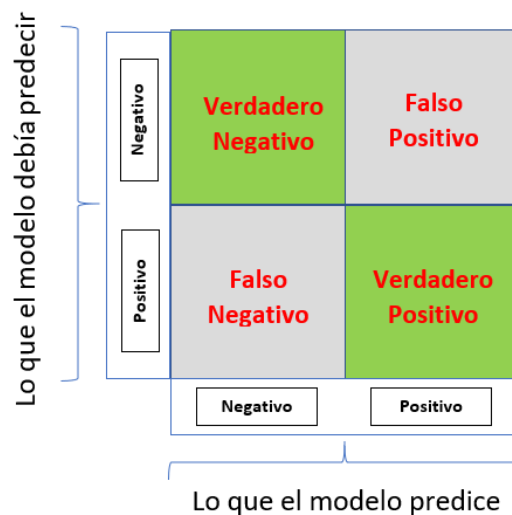


Figura 11: Funcionamiento de matriz de confusión

Ya comprendido lo anterior se presentan las métricas con las que se evaluó el proyecto:

5.1 Precision

Sirve para medir la calidad del modelo en tareas de detección, donde se refiere a, la proporción de instancias positivas que han sido clasificadas correctamente respecto al total de instancias clasificadas como positivas, que son verdaderos y falsos positivos. Mientras mayor sea la precisión, mayor confianza tendrá el modelo al clasificar una muestra como positiva (Díaz, 2020). Su fórmula es:

$$\text{Precision} = \frac{TP}{TP+FP}$$

5.2 Recall

Esta métrica, también conocida como sensibilidad, mide la proporción de instancias positivas que son clasificadas correctamente respecto al total de instancias positivas, como verdaderos positivos o falsos negativos. Mientras mayor sea el Recall, más muestras positivas clasificará el modelo correctamente (Díaz, 2020). Su fórmula es:

$$\text{Recall} = \frac{TP}{TP+FN}$$

5.3 F1-Score

Es la media armónica entre la precisión y Recall. Siendo de utilidad cuando se busca un equilibrio entre estas 2 métricas mencionadas. Puesto que, si el Recall es alto y la precisión baja, existe la probabilidad de tener mayor cantidad de falsos positivos, mientras que, si el Recall es más bajo y la precisión más alta, entonces el modelo será preciso cuando clasifica una muestra como positiva. Midiendo el equilibrio entre la precisión y recuperación, donde un F1 alto indica que la precisión y el Recall son altos, mientras que un F1 bajo implica un mayor desequilibrio en estos valores (Díaz, 2020). Su fórmula es:

$$F1 = 2 * \frac{PRECISION * RECALL}{PRECISION+RECALL}$$

5.4 IoU

Es la intersección sobre la unión, que define la cantidad de superposición entre el cuadro delimitador predicho y el real. Por ejemplo, un ioU igual a 0, quiere decir que no existe superposición entre las casillas, mientras que un IoU de 1, significa que la unión de las casillas entre el cuadro delimitador predicho y el real es perfecto y totalmente superpuesto (Díaz, 2020). Su fórmula es:

$$\text{IoU} = \frac{\text{Área de intersección}}{\text{Área de Unión}}$$

5.5 mAP

Esta métrica hace alusión a la precisión promedio, donde, en principio, según el conjunto de datos, se calcula el IOU según cada cuadro etiquetado. Para luego generar una curva de precisión versus el Recall para cada clase, obteniendo así la precisión promedio (AP). Representando el rendimiento de un modelo con respecto a los verdaderos positivos, falsos positivos y falsos negativos en un determinado rango de valores de confianza. Donde el mAP es el promedio del AP de todas las clases (Díaz, 2020). Su fórmula es:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^n AP_i$$

6. Herramientas

6.1 Lenguajes de Programación

6.1.1 Python

Es un lenguaje de programación de alto nivel, famoso por su sintaxis clara y legible. Siendo muy popular en la comunidad de la ciencia de datos e inteligencia artificial debido a su gran versatilidad y amplia gama de bibliotecas disponibles que faciliten considerablemente la implementación de algoritmos. Su versión actual, con la que se trabajará en el proyecto es la 3.12.0 (*Python, s.f*).



Figura 12: Logotipo de Python

6.1.2 Dart

Lenguaje de programación moderno y de código abierto desarrollado por Google. Diseñado con el objetivo de permitir un desarrollo estructurado y escalable de aplicaciones web móviles y de escritorio. Este lenguaje está basado en clases y orientado a objetos, con una sintaxis familiar a lenguajes populares como Java y #C, siendo una gran alternativa para programadores por sus características de versatilidad y eficiencia, ofreciendo un gran equilibrio entre facilidad de desarrollo y alto rendimiento (*Dart, s.f*).



Figura 13: Logotipo de Dart

6.2 Bibliotecas

6.2.1 OpenCV

Es una biblioteca de visión por computadora y aprendizaje automático de código abierto, que proporciona herramientas para manejar el procesamiento de imágenes y videos, permitiendo capturar y analizar cuadros de cámaras en tiempo real, siendo sumamente funcional para tareas de detección de objetos y seguimientos de movimiento (*OpenCV, 2019*).



Figura 14: Logotipo de OpenCV

6.2.2 Pytorch

Es una biblioteca de aprendizaje automático de código abierto para Python, característica por su flexibilidad, eficiencia y diseño amigable. Es utilizada para el desarrollo de sistemas de inteligencia artificial, en el campo del aprendizaje profundo, ayudando a crear modelos y entrenarlos por sus características para manejar redes neuronales dinámicas (*Pytorch, s.f-b*).



Figura 15: Logotipo de Pytorch

6.2.3 Matplotlib

Esta biblioteca está orientada a la representación de datos en Python, permitiendo crear figuras y gráficos representativos en una variedad de formatos estáticos, animados e interactivos. Siendo útil en la visualización de datos y resultados de análisis del procesamiento de imágenes y detección de objetos, lo cual es esencial para entender de mejor manera el desempeño y precisión de modelos (*Matplotlib, s.f*).



Figura 16: Logotipo de MatplotLib

6.2.4 **Ultralytics**

Es una empresa que ofrece una implementación de la arquitectura YOLO, la cual es una herramienta para la detección de objetos en tiempo real. Se ofrecen variadas versiones actualizadas de la CNN YOLO, donde cada versión ofrece aún más subversiones orientadas a distintos escenarios de utilidad (*Ultralytics, s.f*).



Figura 17: Logotipo de Ultralytics

6.2.5 **NumPy**

Esta biblioteca es fundamental para computación científica en Python, proporcionando soporte para el uso de arreglos y matrices de gran tamaño y funciones matemáticas de alto nivel para la operación de datos (*Numpy, s.f*).



Figura 18: Logotipo de NumPy

6.2.6 **pandas**

Es una biblioteca escrita para el lenguaje de programación Python para la manipulación y el análisis de datos. Ofrece estructuras de datos y operaciones para manipular tablas numéricas y series temporales. Siendo de utilidad en el manejo de datos relacionados a la detección, estadísticas de resultados y métricas que se manejen y requieran analizar (*pandas, s.f*).



Figura 19: Logotipo de Pandas

6.3 Base de Datos

6.3.1 MySQL

Es un sistema de gestión de base de datos relacional de código abierto, conocido por su robustez, escalabilidad, y cumplimiento de los estándares SQL. Es bastante usado en el área de manejo de datos por sus características confiables y eficaces, además de contar con una gran seguridad, integridad, atomicidad, durabilidad y consistencia (MySQL, s.f-c).



Figura 20: Logotipo de MySQL

6.4 Etiquetado

6.4.1 LabelImg

Es una herramienta de anotación gráfica de código abierto utilizada para etiquetar imágenes para tareas de aprendizaje automático, especialmente el campo de la visión por computadora. Es popular por su simplicidad y eficacia en la creación de etiquetas para la formación de conjuntos de datos, que tienen como objetivo entrenar modelos de inteligencia artificial para la detección de objetos. Además, posee una amplia gama de formatos entre los que destaca el formato YOLO (LabelImg, s.f).



Figura 21: Logotipo de LabelImg

6.5 Entornos de Desarrollo

6.5.1 Visual Studio Code

También conocido como VS Code, es un editor de código fuente desarrollado por Microsoft, el cual se destaca por ser una herramienta poderosa y versátil utilizada por programadores para una gran variedad de tareas de desarrollo de software. Teniendo la capacidad de manejar una serie de lenguajes y extensiones para que los usuarios puedan trabajar en él, y todo esto en una interfaz sencilla e intuitiva (*VisualStudio, s.f-d*).



Figura 22: Logotipo de Visual Studio Code

6.5.2 Google Colaboraty

Este servicio de Google permite a los usuarios escribir y ejecutar código en Python en un navegador web de forma gratuita. Es popular entre usuarios dedicados al desarrollo de modelos de inteligencia artificial por su sincronización en el trabajo colaborativo con más usuarios, además de presentar un sistema sencillo y de fácil uso para programar (*Google colab, s.f*).



Figura 23: Logotipo de Google Colab

6.5.3 Flutter

Es un framework de código abierto creado por Google para el desarrollo de aplicaciones móviles. El cual se ha ganado una gran reputación producto de su capacidad para permitir a los desarrolladores construir aplicaciones de alta calidad para Android como para iOS con un único código base. Utiliza el lenguaje Dart, y presenta una serie de beneficios a la hora de su uso, como su rendimiento nativo, su función de observar cambios en tiempo real, y su interfaz atractiva y variada (*Flutter, s.f*).



Figura 24: Logotipo de Flutter

6.6 Hardware

6.6.1 Cámara

Para el desarrollo del proyecto, se han ocupado cámaras ya instaladas en las dependencias de la Universidad del Bío-Bío, Chillán. Estas cámaras son de la marca Hikivision y presentan una resolución de 2 megapíxeles a una velocidad de 30 fotogramas por segundo.



Figura 25: Cámara de seguridad Hikivision

Ambas etapas mencionadas hasta el momento ya han sido abordadas en el informe, puesto que la etapa 1 “Entendimiento del negocio” ha sido estudiada y planteada en la introducción y descripción del problema, siendo correctamente justificada. Y la etapa 2 “Entendimiento de los datos” ha sido abordada gracias a la revisión sistemática de la literatura, donde se logró comprender qué tipo de datos usar y cómo manejarlos, además de técnicas para procesarlos y limpiarlos.

7.3 Etapa 3 “Obtención y preparación de los datos”

En esta etapa, se describe detalladamente cómo se seleccionaron y procesaron los datos necesarios para el proyecto. Se aborda desde la fuente y tipo de datos hasta las técnicas de limpieza y transformación utilizadas, explicando de qué manera aportaron al resultado final previo al modelado para asegurar una comprensión clara de la preparación de los datos para su uso en el modelo (*Haya P, 2021*).

Para el desarrollo de un modelo de red neuronal convolucional, se destacan dos partes fundamentales que lo componen. La primera, es la arquitectura, que supone los algoritmos que conforman las capas. Y la segunda, es el dataset, que son un conjunto de datos (imágenes en este caso) que permitirán el aprendizaje o entrenamiento de la red, para que pueda reconocer y clasificar objetos. Además de servir para evaluar el rendimiento del modelo.

Paso N°1: Uso de dataset COCO

Al ser importada las arquitecturas (YOLO), estas vienen preentrenadas, haciendo uso de un dataset popular, llamado COCO (Common Objects in Context), el cual está conformado por 330.000 imágenes recopiladas de diversas fuentes, con el objetivo de representar escenarios de la vida real y objetos cotidianos, siendo estos etiquetados con una respectiva clase, que generaliza a dicho objeto (COCO, 2023). Para que YOLO pueda usar el conjunto de datos, se debe tener en la carpeta del proyecto un archivo COCO.txt, que contenga las clases que define los objetos con que se etiquetaron las imágenes. Existen variadas clases con su respectivo número, como por ejemplo la clase 3 corresponde a car, como se muestra en la *Figura 27*.

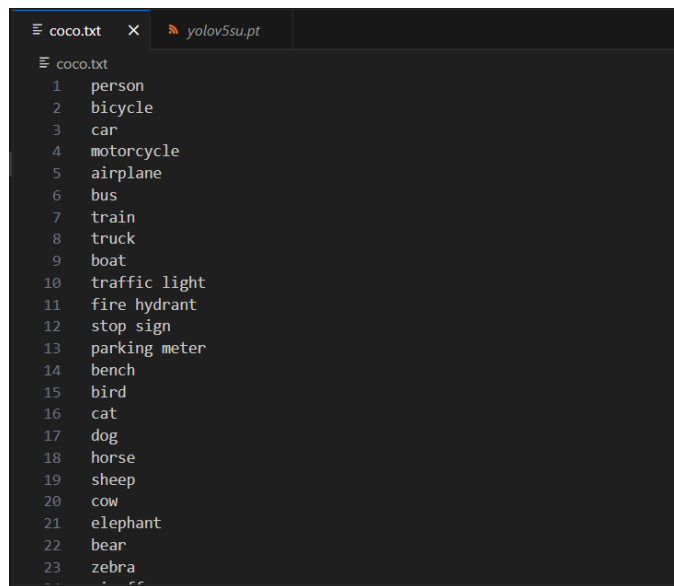


Figura 27: Clases del dataset COCO

Paso N°2: Obtención de imágenes para el dataset

Al realizar lo anterior y contar con el dataset COCO, el modelo logra detectar una serie de objetos de manera efectiva, como por ejemplo vehículos. Sin embargo, se estima que el modelo puede mejorar aún más si se entrena con imágenes del lugar objetivo donde se implementará, siendo este el estacionamiento seleccionado en la Universidad del Bío-Bío mencionado anteriormente. Esto porque estará familiarizado con el entorno y sus características, aprendiendo la posición en que los vehículos suelen estacionarse.

Se define la creación de dos conjuntos de datos, conformados por las mismas imágenes, pero con diferentes enfoques que pueden dar solución al problema. El primero se creará para la detección de vehículos en el estacionamiento. Y el otro se creará para la detección de cupos libres y ocupados en el estacionamiento. Donde finalmente se compararán, y se seleccionará el que tenga un mejor rendimiento para la detección de plazas libres y ocupadas.

Dado lo anterior se solicitó un conjunto de grabaciones del estacionamiento, obtenidas de una cámara situada en una esquina del edificio de los laboratorios centrales. Se han escogido 5 días de grabaciones capturando imágenes cada 20 segundos, obteniendo un total de 1300 screenshots. Cabe destacar un problema con la posición de la cámara, se puede observar en la *Figura 28* que presenta complejidades en base al ángulo de grabación que tiene, puesto que, al encontrarse en una esquina, permite que los vehículos más alejados, se vean solapados por los que tienen delante, dejándolos prácticamente indetectables al solo verse una pequeña o ninguna parte de estos.



Figura 28: Posición y ángulo de la cámara al estacionamiento

Paso N°3: Limpieza de imágenes

Una vez se tienen las imágenes del estacionamiento, se debe efectuar una limpieza, donde se revisa que imágenes sirven y cuáles se eliminan, basándose en ciertos criterios de eliminación como:

- El ruido presente en las imágenes, el cual debe ser mínimo, para que, de estos datos, la arquitectura pueda rescatar características de los objetos sin problemas, minimizando el índice de confusión. Como se muestra en la *Figura 29*.
- La similitud de imágenes, donde estos datos que conforman el dataset, no deben ser muy similares, puesto que lo ideal es tener un conjunto de datos diverso. Imágenes iguales, pueden generar sobreajuste y ser información redundante en el conjunto. Como se muestra en la *Figura 30*.





Figura 30: Similitud de imágenes, con selección de la primera y eliminación de la segunda

Con lo anterior realizado se reduce el número total de datos, a 800 imágenes que serán procesadas en el siguiente punto.

Paso N°4: Definición de clases para el etiquetado

Con el conjunto de imágenes normalizado, se procedió a realizar el etiquetado. Para lo cual se hizo uso de la herramienta Labellmg. Lo primero fue definir las clases que contendrá cada dataset para el etiquetado, y que lo tendrá cada uno en un archivo `classes_predefined.txt`. Creándose así las respectivas clases, uno enfocándose en la detección de vehículos y el otro enfocándose en la detección de espacios libres y ocupados, quedando como se muestra en la Figura 31 y la Figura 30 respectivamente.

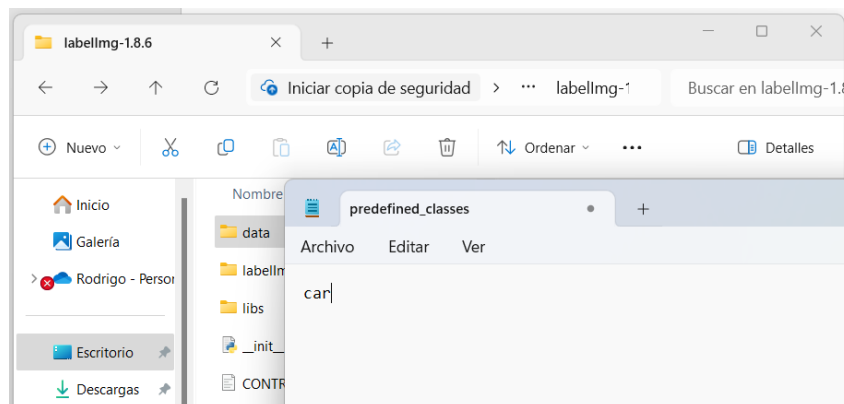


Figura 31: Clase para el modelo de detección de vehículos

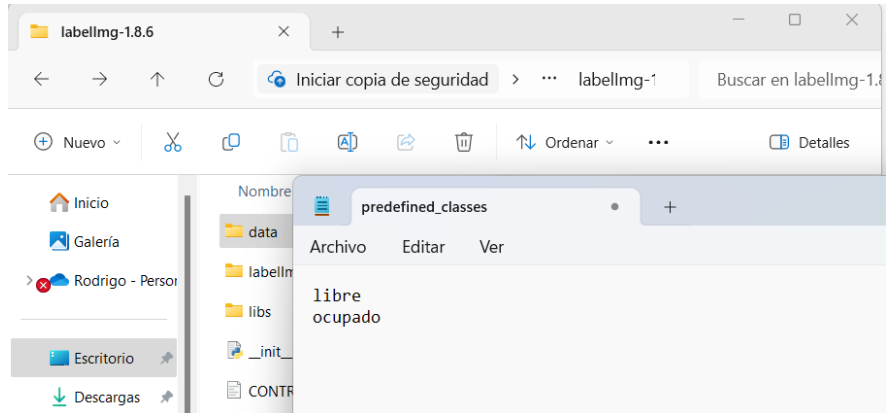


Figura 32: Clases para el modelo de detección de espacios

Paso N°5: Etiquetado de imágenes

Luego se carga la carpeta de las imágenes a LabelImg y se realiza el etiquetado, donde el primer paso es definir el tipo de formato deseado, siendo YOLO en ambos casos. Y definiendo el dataset de espacios ocupados y vacíos como “EspaciosUBB” y el de vehículos “VehículosUBB”. Después solo queda crear los Bounding-boxes (cajas delimitadoras), para que cada objeto se defina en la imagen, asociándole la respectiva clase a la que pertenece. Al igual que se observa en la Figura 33 y la Figura 34.



Figura 33: Etiquetado de imágenes para el dataset VehiculosUBB

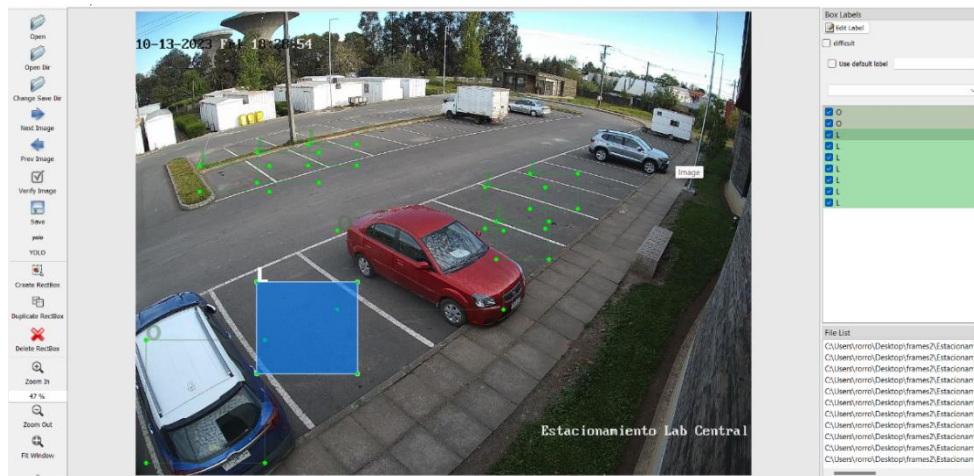


Figura 34: Etiquetado para el dataset de EspaciosUBB

Para el etiquetado se definen ciertos criterios:

- Los bounding box deben intentar no superponerse.
- Los bounding box deben encerrar solo 1 auto o 1 solo espacio dependiendo de la clase (aunque sea solo una parte).
- Se deben encerrar cada uno de los vehículos del estacionamiento y los espacios que se deseen marcar (clasificarlos todos).
- Los bounding box en el caso de la clase de espacios deben encerrarse cuando se noten los límites de las respectivas líneas blancas del suelo.

Ya etiquetadas todas las imágenes, se tienen los conjuntos de datos listos para ser usados en el entrenamiento del modelado. Donde cada uno tiene los datos ordenados en una carpeta que agrupa las imágenes y otra que agrupa los labels en formato YOLO, quedando así para poder cargar de manera correcta los dataset al entorno de programación.

7.4 Etapa 4 Modelado

En esta etapa se presenta el modelado de la red neuronal convolucional, en donde se trabajó con los datos que fueron procesados anteriormente. Se procedió a definir la arquitectura y finalmente a entrenarla.

La arquitectura que se seleccionó para desarrollar el modelo es YOLO, por variadas razones, como, su velocidad, que lo hace ideal para implementaciones en tiempo real, su precisión al detectar objetos, su generalización al ser preentrenado y su forma de detección en una sola pasada a través de la imagen. Los modelos en cuestión deben estar preentrenados, puesto que eso implica que ya han aprendido características generales en conjuntos de datos grandes y variados, lo que posteriormente permitirá al modelo converger más rápidamente que un modelo entrenado desde cero, lo que ahorra tiempo y recursos computacionales. Además, al ya tener características aprendidas no requiere de un dataset robusto. Así solo se deben entrenar con las clases que se requieran detectar y realizar los ajustes de parámetros que se requieran.

Esta arquitectura cuenta con 8 versiones actualmente, desde YOLOv1 hasta YOLOv8. Donde se presentan mejoras en rendimiento, adaptabilidad, capacidad de detección, entre otras características. Además, desde la versión 5 en adelante presentan subversiones que varían dependiendo de sus parámetros y capas, las cuales son las subversiones “n” - “s” - “m” - “l” - “x”

Paso N°1: Preparación del entorno de programación.

El primer paso realizado, fue la creación de un environment, el cual es un nuevo entorno aislado que puede tener diferentes versiones de bibliotecas o lenguajes de programación, sin que exista conflicto. La razón de esta acción está dada por problemas al trabajar con la biblioteca PyTorch en versiones recientes del lenguaje Python, en específico para este proyecto se utilizó la versión de Python 3.11.0 y la versión 2.1.0+cu121 de PyTorch.

Después se creó un archivo Python, donde se importaron las diferentes bibliotecas que se usaron, como, OpenCV para manejar los videos e imágenes, PyTorch para trabajar con las capas que componen la red y utilizar la plataforma CUDA que permite el uso del GPU, Ultralytics para importar al modelo de YOLO preentrenado, Pandas para manejar datos y NumPy para trabajar con arreglos de datos.

Se utilizó la biblioteca Ultralytics para cargar el modelo YOLO preentrenado, definiendo la versión que se desea importar. Luego se definió una función que era la encargada de entrenar el modelo. Para esto se necesitan dos parámetros, siendo el primero un archivo tipo Yaml que contendrá tanto la ruta hacia el conjunto de datos, con sus respectivas carpetas de entrenamiento y validación como la descripción de la cantidad de clases que reconoce el modelo. Como se muestra en la *Figura 35*.

```

! config.yaml
1 path: C:\Users\josse\OneDrive\Escritorio\dataset2.0
2 train: C:\Users\josse\OneDrive\Escritorio\dataset2.0\train\images
3 val: C:\Users\josse\OneDrive\Escritorio\dataset2.0\value\images
4
5
6 names:
7   0: car

```

Figura 35: Ruta y clases para la importación de YOLO

Paso N°2: Arquitecturas entrenadas

Con el entorno de programación ya conformado, se dispuso a realizar cada uno de los entrenamientos con dos versiones diferentes de YOLO. Escogiendo la versión YOLOv5 por ser una de las más populares, por su fácil nomenclatura, facilidad de uso y sus constantes actualizaciones en velocidad y precisión hasta día de hoy. Junto a la versión de YOLOv8, puesto que es la más reciente. Donde se compararán sus respectivas subversiones “n” – “s” – “m” en cada una. Se ha declinado el uso de YOLOv6 y YOLOv7 en la comparativa, puesto que no presentan modelos preentrenados para importar.

A continuación, en la *Figura 36* y la *Figura 37*, se observa la comparativa del rendimiento entre las distintas subversiones de cada versión de YOLO con que se trabajó (Las versiones con extensión .pt son modelos preentrenados y con pesos definidos, los modelos con la extensión .yaml son versiones sin preentrenamiento).

Aunque las versiones más reducidas presenten resultados promedios menores que sus pares más complejos, esto no quiere decir que funcionen mejor para el objetivo propuesto, ya que existe dependencia de múltiples factores, como parámetros, conjunto de datos, entre otros. Por lo cual se decide realizar las respectivas pruebas para corroborar la información.

Detection

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include 80 pre-trained classes.

Model	YAML	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
yolov5nu.pt	yolov5n.yaml	640	34.3	73.6	1.06	2.6	7.7
yolov5su.pt	yolov5s.yaml	640	43.0	120.7	1.27	9.1	24.0
yolov5mu.pt	yolov5m.yaml	640	49.0	233.9	1.86	25.1	64.2
yolov5lu.pt	yolov5l.yaml	640	52.2	408.4	2.50	53.2	135.0
yolov5xu.pt	yolov5x.yaml	640	53.2	763.2	3.81	97.2	246.4

Figura 36: Ultralytics. (n.d). Resultados promedios de las versiones de arquitecturas de YOLOv5 <https://docs.ultralytics.com/models/yolov5/#key-features>

Detection (COCO)	Detection (Open Images V7)	Segmentation (COCO)	Classification (ImageNet)			
See Detection Docs for usage examples with these models trained on COCO , which include 80 pre-trained classes.						
Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Figura 37: Ultralytics. (n.d). Resultados promedios de las versiones de arquitecturas de YOLOv8 <https://docs.ultralytics.com/models/yolov5/#key-features>

Se busca encontrar la arquitectura que presente el mejor rendimiento para el caso y entorno abordado. Estas presentan diferencias en su estructura, como, funciones, cantidad de capas y parámetros.

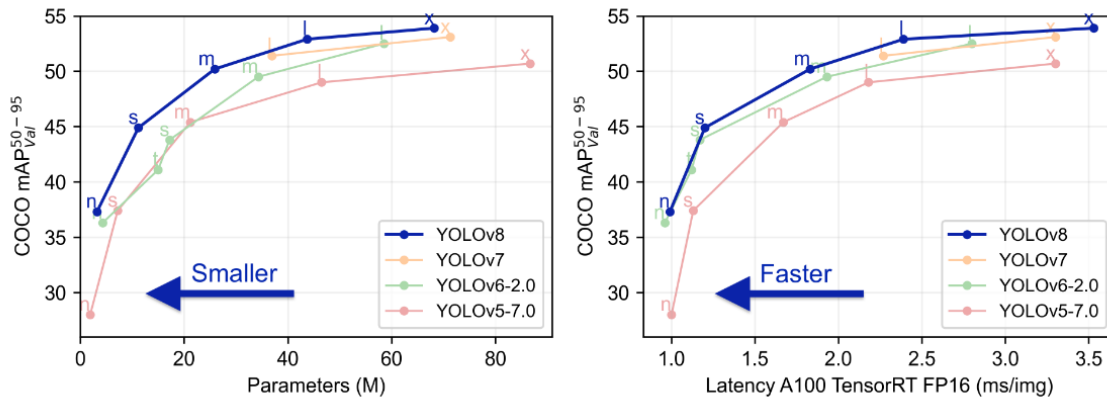


Figura 38: Ultralytics. (n.d). Comparativa general de precisión entre las versiones de YOLO <https://docs.ultralytics.com/models/yolov8/>

En la Figura 38 se observa que las variantes más antiguas muestran menor precisión, aunque eso no demuestra el comportamiento que puede tener un modelo específico, puesto que las arquitecturas con menos parámetros tienden a funcionar mejor con datasets pequeños, mientras que las arquitecturas más complejas, funcionan mejor con datasets más grandes.

Paso N°3: Técnicas utilizadas

En el modelado se utilizan una serie de técnicas con diferentes objetivos, que se implementaron en el modelo para mejorar su rendimiento y manejo.

- **Early Stopping:** Es una técnica de regularización, utilizada para evitar el sobreajuste durante el entrenamiento del modelo. Dicho sobreajuste sucede cuando un modelo, se ajusta demasiado bien a los datos del entrenamiento, se le dificulta generalizar de manera correcta nuevos datos que sean diferentes. Entonces se monitorea el rendimiento y si en el conjunto de test el error de validación comienza a aumentar y el modelo ya no aprende, el entrenamiento se detiene para evitar el sobreajuste (Keras, s.f).
- **DropOut:** Esta técnica sirve para evitar el sobreajuste de un modelo. Esto por medio de la desactivación aleatoria de neuronas que componen las capas del modelo, lo cual se hace con un porcentaje de apagado definido. Hace que la red neuronal, se acostumbre menos a activaciones específicas de las neuronas, reduciendo el riesgo de dependencia excesiva en un pequeño número de neuronas, lo que lleva al sobreajuste (Keras, s.f).
- **Region of Interest (ROI):** Es una técnica relacionada al aspecto de visión por computadora, donde se selecciona una determinada área de interés en una parte específica de la imagen en la que se centrará el algoritmo que procesa la imagen. Es de ayuda para reducir el área de búsqueda, para mejorar el rendimiento del modelo, en aspectos de tiempo y gasto computacional, ya que se reduce el lugar de búsqueda para detectar objetos (Region of interest, 2020).

Paso N°4: Entrenamiento de los modelos

Se realizó el entrenamiento de cada arquitectura a comparar, con tal de enseñarle a esta a reconocer características y patrones. Para esto, se hizo uso de la técnica del transfer Learning, que se refiere al hecho de aprovechar el conocimiento de un modelo preentrenado para adaptarlo a nuevas tareas, con el uso de datasets pequeños. Además, se hizo uso del fine tuning, que es una técnica que consiste en realizar un ajuste de capas a un modelo preentrenado para adaptarlo así, a la tarea específica que se desea realizar. Esto se hizo ajustando las capas de salida reduciendo su dimensionalidad a 2 clases, mientras que las capas iniciales se mantienen con los pesos del preentrenamiento.

En la *Figura 39*, se detalla la función para realizar el entrenamiento con cada modelo importado de YOLO. Donde se utilizan 2 dataset compuestos por 800 imágenes etiquetadas cada uno. Donde el primero se utilizó para probar el modelo en la detección de autos y el segundo para probar el modelo en la detección de espacios (libre/ocupado).

```

modeltrain.py > ...
1  from ultralytics import YOLO
2
3  model = YOLO(["yolov8n.pt"])
4  def main():
5
6      results = model.train(data="config.yaml", epochs=100)
7
8  if __name__ == '__main__':
9      main()
10

```

Figura 39: Entrenamiento de un modelo de 100 épocas

A cada modelo se le ajustaron determinados hiperparámetros, los cuales se pueden observar en la *Figura 40*. Definiendo un total de 100 épocas de entrenamiento, que es una cantidad suficiente para que los modelos aprendan. Una tasa de aprendizaje de 0.0001, la cual debe ser baja para no perturbar drásticamente los pesos que posee el modelo preentrenado y un batch size de 32, el cual corresponde a la cantidad de entrenamientos en una iteración, el cual depende del tamaño del dataset y del poder computacional. Y finalmente el uso de la función de DropOut al 40%, donde solo se usarán aleatoriamente, el 60% de las neuronas totales para el train, con el fin de evitar el sobreajuste.

```

Hiperparametros.py > main
1  from ultralytics import YOLO
2
3  def main():
4      model = YOLO('yolov8s.pt')
5
6      batch_size = 32
7      learning_rate = 0.0001
8      num_epochs = 100
9      por_dropout = 0.4
10
11     results = model.train(data='config.yaml',
12                           batch=batch_size,
13                           lr0=learning_rate,
14                           epochs=num_epochs,
15                           dropout=por_dropout)
16
17  if __name__ == '__main__':
18      main()
19

```

Figura 40: Ajuste de hiperparámetros

Al realizar estos entrenamientos con las diferentes subversiones de las arquitecturas de YOLOv5 y YOLOv8, ya se tiene completo el modelado, con una arquitectura que debe saber reconocer las clases con las que fue etiquetado y los objetos con los cuales se entrenó.

Luego de esto, el modelo se somete a pruebas de validación para comprender el comportamiento de su aprendizaje, para saber cuánto y cuando aprendió. Midiendo estos datos, bajo métricas de evaluación implementadas y definidas en el modelado las cuales se guardan en una ruta específica.

Paso N°5: Estructuras de los modelos

Se presentan estructura de la red neuronal que conforman los modelos, donde todos presentan las siguientes capas, variando en cantidad y posición:

- Las capas de convolución son las encargadas de extraer las características espaciales de las imágenes.
- Las capas de BatchNorm, normalizan la salida de las capas de convolución para mejorar el entrenamiento.
- La función de activación SiLu, introduce la no linealidad, permitiendo al modelo aprender patrones complejos.
- Los bloques C3, combinan capas de convolución para así mejorar la capacidad del modelo para aprender características jerárquicas.
- La capa SPPF es como una capa de agrupamiento, que facilita el manejo de imágenes y mejora la capacidad del modelo para detectar objetos en escalas variadas.
- Las capas de concatenación combinan información de variadas escalas de la red, integrando características de diferentes niveles.
- Y la capa de detección, que utiliza las características aprendidas para realizar la detección de objetos, predicción de cajas delimitadoras, clases y puntuaciones de confianza.

La estructura de cada versión se puede resumir en las imágenes de las *Figura 41* y *Figura 42*, donde mediante un algoritmo se contabiliza la cantidad de capas, y se comprende la variación en la composición de cada una.

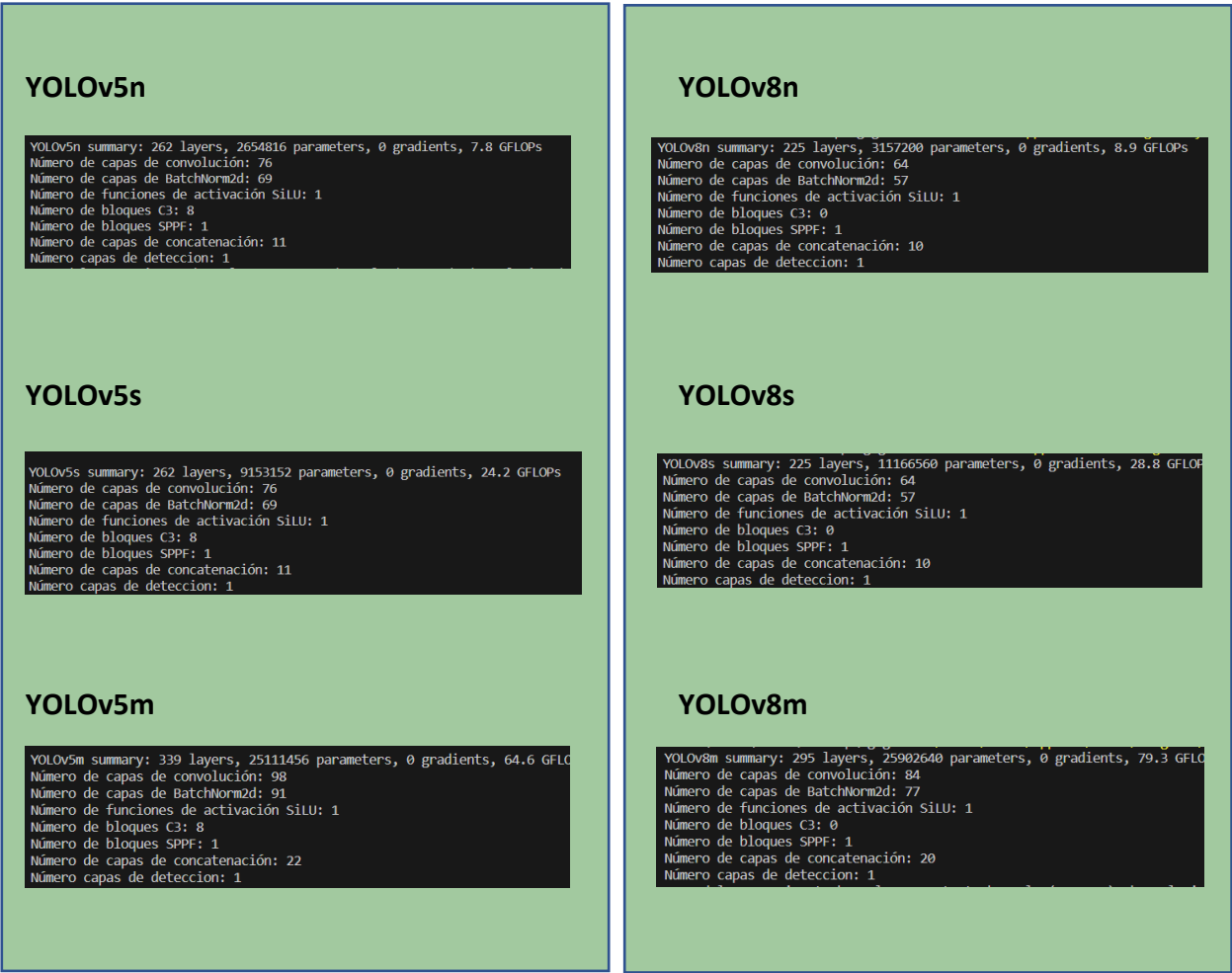


Figura 42: Cantidad de capas de las versiones de YOLOv5 *Figura 41: Cantidad de capas de las versiones de YOLOv8*

7.5 Evaluación

Una vez que los modelos fueron construidos, se procedió a una evaluación meticulosa utilizando métricas fundamentales como la Precisión, el Recall y el F1-Score. Estas fórmulas matemáticas proporcionan una interpretación precisa de la calidad del modelo, revelando el porcentaje de éxito en relación con la tarea asignada.

La elección de evaluar seis arquitecturas distintas se basa en la necesidad de explorar a fondo las capacidades de detección de los modelos. En particular, se consideraron tres subversiones de YOLOv5 y tres subversiones de YOLOv8. Esta elección se fundamenta en la reputación y eficacia demostrada de ambas versiones de YOLO en la detección de objetos en tiempo real.

La razón de incluir subversiones específicas de YOLOv5 y YOLOv8 radica en sus características diferenciales y avances tecnológicos. YOLOv5 se ha destacado por su capacidad para lograr un equilibrio entre precisión y velocidad, siendo una elección óptima para aplicaciones en tiempo real. Por otro lado, YOLOv8, al ser una versión más reciente, ha introducido mejoras y ajustes que podrían influir en su rendimiento.

La elección de utilizar dos conjuntos de datos distintos, uno centrado en la detección de espacios vacíos u ocupados (caso 1) y otro orientado a la detección de vehículos (caso 2), se justifica por la naturaleza específica de la problemática en la Universidad del Bío-Bío. Dicha diversidad de conjuntos de datos permite abordar las necesidades concretas de detección en los escenarios de estacionamiento, lo que facilita la identificación del modelo más adecuado para resolver cada situación.

Como justificación, la selección de estas versiones específicas de YOLO se basa en su probada eficacia en la detección de objetos y su relevancia en aplicaciones en tiempo real. La diversificación de subversiones y la elección de conjuntos de datos específicos proporcionan una evaluación integral, permitiendo la identificación del modelo más adecuado para abordar las problemáticas específicas de la Universidad del Bío-Bío.

7.5.1 Caso 1: Resultados de Dataset “EspaciosUBB”

- **Uso de YOLOv5**

En principio, se comparó las tres subversiones de la arquitectura YOLOv5, mediante un gráfico que muestra el rendimiento de los modelos entrenados con el mismo dataset, etiquetado con dos clases, que son espacios libres y ocupados.

Se muestra el gráfico de la *Figura 43*, que mide el mAP (eje X) a un IoU de 0.5 de las tres subversiones, a lo largo de 100 épocas de entrenamiento aplicadas (eje y) con el conjunto de datos en cuestión.

Del gráfico se observa que todas las versiones alcanzan y mantienen un alto mAP después de las 20 épocas, lo cual es un indicativo de que los modelos han aprendido las características extraídas del conjunto de datos y han logrado generalizar efectivamente. En primera instancia se desestima la presencia de sobreajuste en los modelos, ya que no existe un descenso del mAP en el entrenamiento. Se observa que, aunque todos los modelos presentan un rendimiento similar, la versión “n” se demoró más épocas en llegar a dicho valor, producto de la menor complejidad de la red.

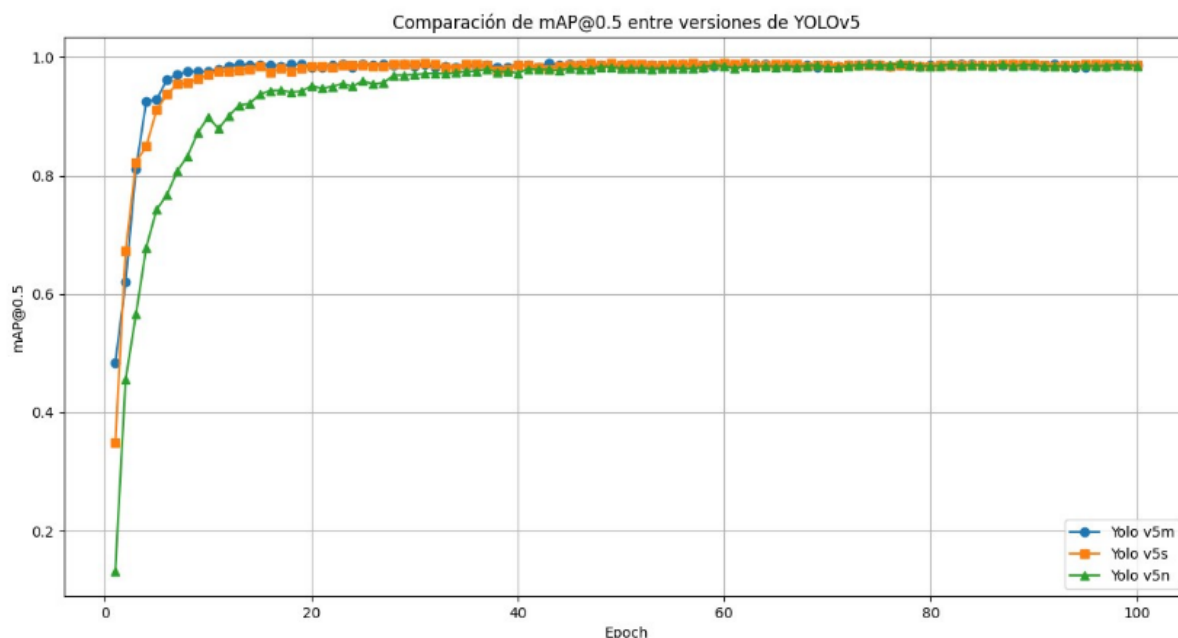


Figura 43: Comparación de mAP de versiones de YOLOv5 con dataset de Espacios

Ya vista la forma en que aprendieron, se necesita discernir que modelo es mejor, por lo cual es necesario observar la pérdida de validación, para saber que tan bien el modelo predice datos que no ha visto durante el entrenamiento. Se observa en la *Figura 44* que el modelo YOLOv5m tiene la pérdida de validación ligeramente más baja en todas las categorías, lo que sugiere que es más eficaz en términos de localización y clasificación de objetos, así como en el aprendizaje de características. Concluyendo en que el mejor modelo es YOLOv5m, por su efectividad al ser la arquitectura más compleja y ser el que presenta mejor rendimiento al detectar los espacios.

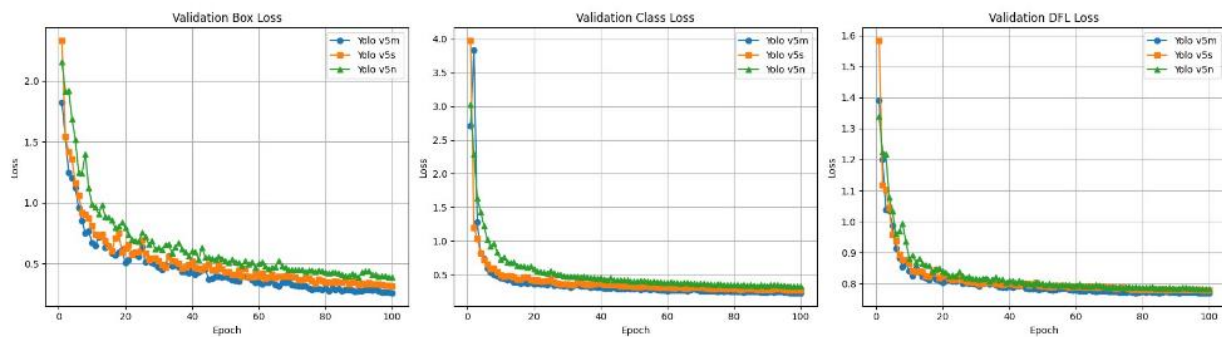


Figura 44: Métricas de validación para entrenamiento de versiones de YOLOv5 con dataset de Espacios

Es importante analizar el desempeño obtenido por cada modelo, que se define a través de una matriz de confusión normalizada de cada uno. La cual explica el rendimiento de cada modelo en la detección de las categorías libre u ocupado, ayudando a entender como este modelo detecta o confunde las clases entre sí.

En la diagonal con pendiente negativa de color azul, se pueden observar valores altos sobre el 97%, correspondientes a los VP y VN, lo que significa que tuvieron un alto rendimiento en la clasificación correcta de ambas clases. Mientras que en la diagonal positiva se observan valores bajos que indican que hay poca confusión entre las clases libre y ocupado que representan los FP y FN.

Por tanto, los modelos son bastante precisos en la diferenciación entre las categorías que están tratando de predecir. Y finalmente, se observa que el modelo de YOLOv5 es el que se comporta ligeramente mejor manera con un 98% de precisión.

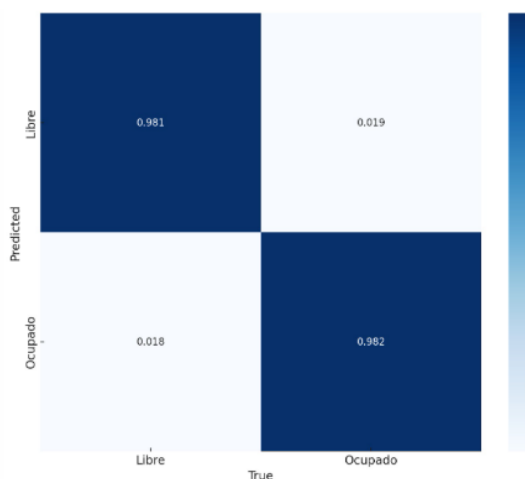


Figura 45: Matriz de confusión normalizada de YOLOv5m

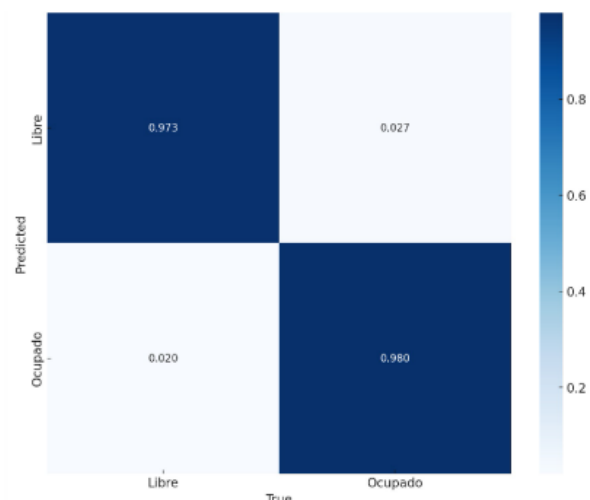


Figura 46: Matriz de confusión normalizada de YOLOv5n

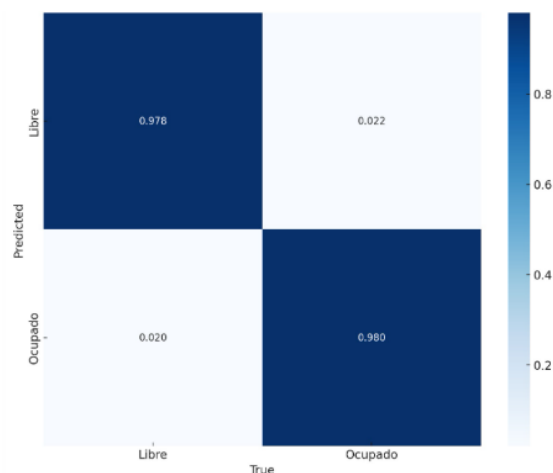


Figura 47: Matriz de confusión normalizada de YOLOv5s

En lo que respecta a la medida del Recall de los modelos, como se muestra en la *Figura 48*, se logra analizar que, en todos los casos, se comienza con un alto Recall, esto indica que se logra una alta detección de instancias positivas correctamente con bajo umbral de confianza. A medida que este umbral sube, el Recall baja, pero gradualmente hasta 0.9 aproximadamente, lo cual indica un buen desempeño. Cayendo finalmente, al tratar imágenes con un alto índice de confianza. Concluyendo en que se logra detectar bien la mayoría de las instancias positivas, pero mostrando dificultades cuando se aumenta el umbral de decisión.

En cuanto al F1-Score, se muestra mayor variación en YOLOv5n, respecto a sus rivales que muestran un rendimiento similar. Donde, en el modelo “n”, la métrica alcanzó valores de 97% cuando el umbral era de 0.5. Mientras que los otros modelos, alcanzaron una puntuación de 98%, con un umbral de confianza de 0.6 aproximadamente. Destacando así el rendimiento de los modelos “s” y “m”. Concluyendo en que al considerar todos los factores YOLOv5m es la mejor opción.

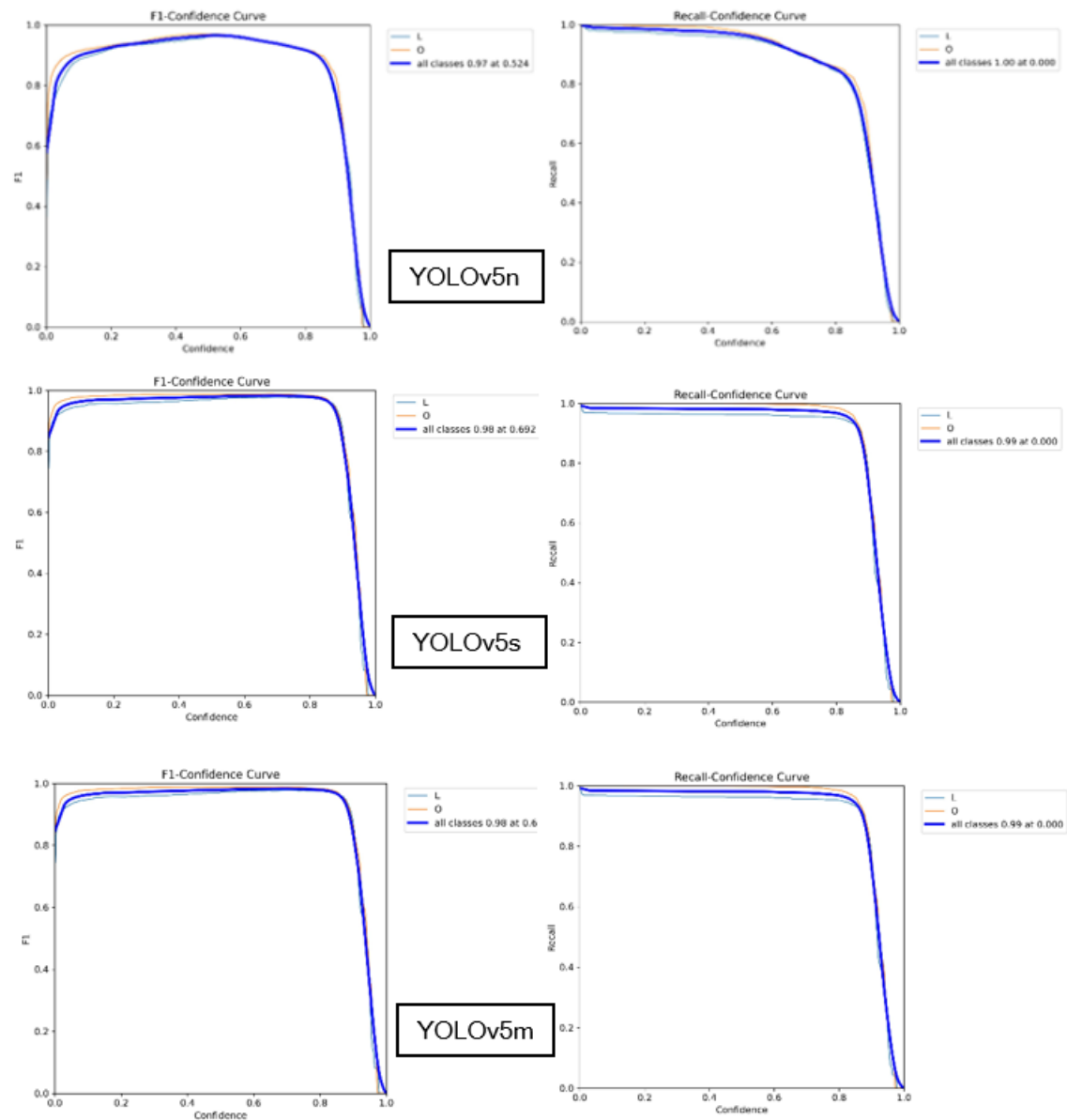


Figura 48: Métricas de F1-Score y Recall de YOLOv5 con dataset "EspaciosUBB"

- **Uso de YOLOv8**

A continuación, se detallan los resultados de las tres subversiones de YOLOv8, bajo las mismas métricas y condiciones de la versión YOLO5 presentada anteriormente. Donde se aprecia que, de manera similar, todas las arquitecturas muestran resultados positivamente parecidos, llegando a valores cercanos al 100%. La versión “n” es la que necesita de más épocas de entrenamiento para lograr resultados favorables, pero al fin y al cabo los logra de manera consistente.

En la *Figura 49* se infiere y destaca que todas las arquitecturas presentan un buen rendimiento, pero es necesario ver la pérdida de validación para comprender de mejor manera sus resultados.

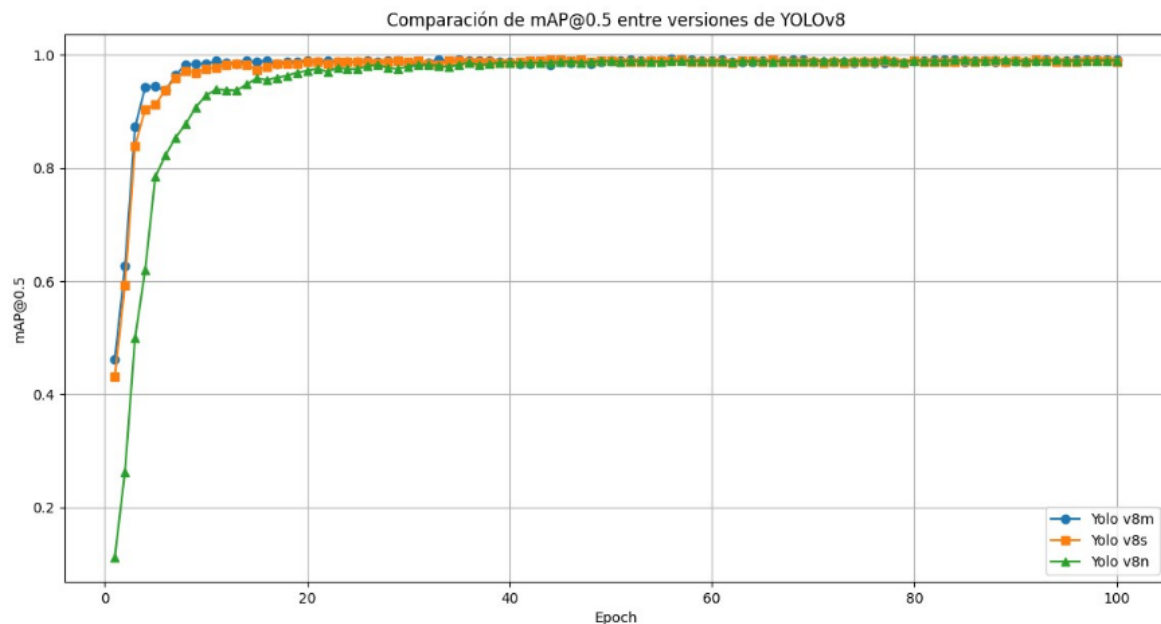


Figura 49: Comparación de mAP de versiones de YOLOv8 con dataset de Espacios

En los gráficos de validación que se presentan en la *Figura 50*, se observa que, aunque los gráficos empiezan con una alta pérdida de validación, conforme transcurre el entrenamiento y se aprende a interpretar los datos, esta disminuye lo que significa que el modelo está mejorando su rendimiento al clasificar los objetos. Las diferencias entre las subversiones son bastante acotadas, pero se aprecia YOLOv8m es la mejor opción puesto que tiene menor pérdida de validación, y presenta menos fluctuaciones en la métrica de validación, concluyendo en un modelo más estable frente a sus rivales.

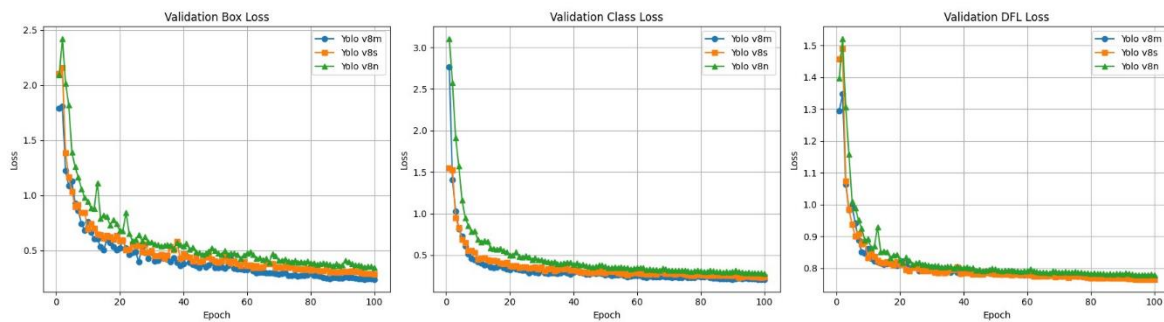


Figura 50: Métricas de validación para entrenamiento de versiones de YOLOv8 con dataset de Espacios

Se comparan las matrices de normalización para medir el porcentaje de rendimiento. Donde se observa que la diagonal con pendiente negativa de color azul posee valores altos sobre el 96%, que corresponden a los VP y VN, lo que significa que tuvieron un alto rendimiento en la clasificación correcta de ambas clases. Mientras que, en la diagonal positiva, se identifican valores bajos, que indican que hay poca confusión entre las clases libre y ocupado que representan los FP y FN.

Concluyendo así que el mejor modelo, que representa el mejor porcentaje de rendimiento al clasificar correctamente las clases es la subversión “m” de YOLOv8, lo cual se puede deber a la mayor complejidad en su estructura que las otras subversiones, permitiéndole rescatar mejor las características y por ende mayor generalización.

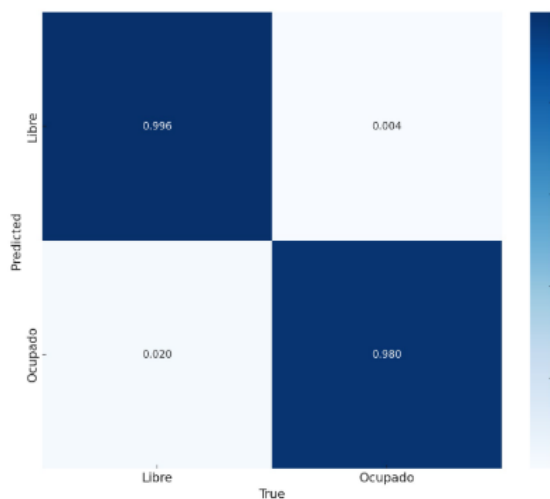


Figura 52: Matriz de normalización de YOLOv8m

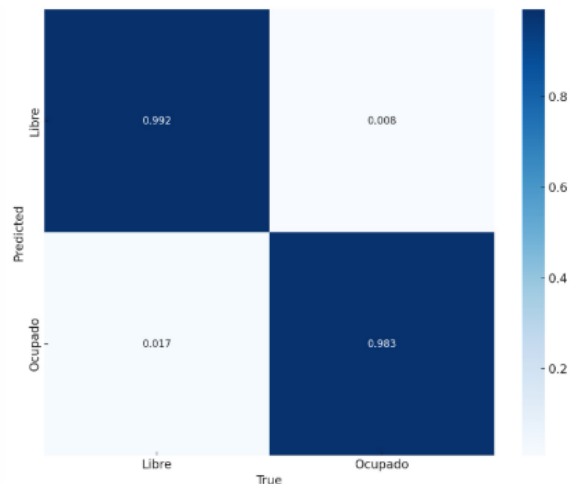


Figura 51: Matriz de normalización de YOLOv8s

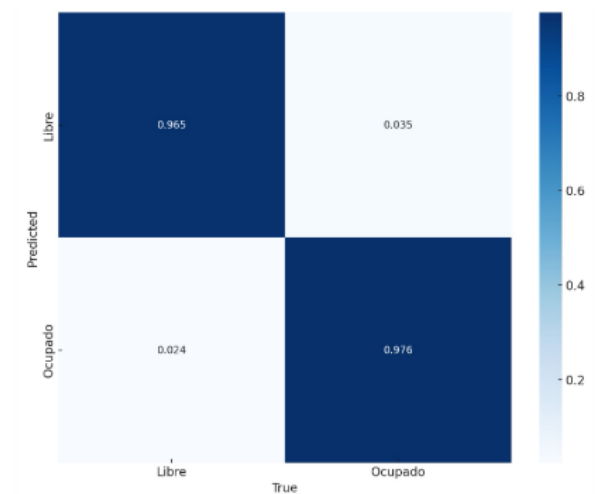


Figura 53: Matriz de normalización de YOLOv8n

La métrica Recall de los modelos representada en la *Figura 54*, destaca que, de manera similar al caso anterior, en todos los casos se comienza con un alto Recall, lo que indica que se logra una alta detección de instancias positivas correctamente, con bajo umbral de confianza. A medida que este umbral sube, el Recall baja, pero gradualmente hasta 0.9 aproximadamente, mostrando mejor rendimiento en los modelos “s” y “m”. Donde finalmente, desciende al tratar las imágenes con un alto índice de confianza. Concluyendo en que se logra detectar bien la mayoría de las instancias positivas, pero mostrando dificultades cuando se aumenta el umbral de decisión.

En cuanto al F1-Score, igualmente al caso anterior, se observa mayor variación en YOLOv5n en comparación a sus rivales, que muestran un rendimiento idéntico. En el modelo “n”, la métrica alcanzó valores de 98% cuando el umbral era de 0.6. Mientras que los otros modelos, alcanzaron igualmente una puntuación de 98%, resaltando el modelo “m” por hacerlo con el mayor umbral de confianza de 0.77 aproximadamente.

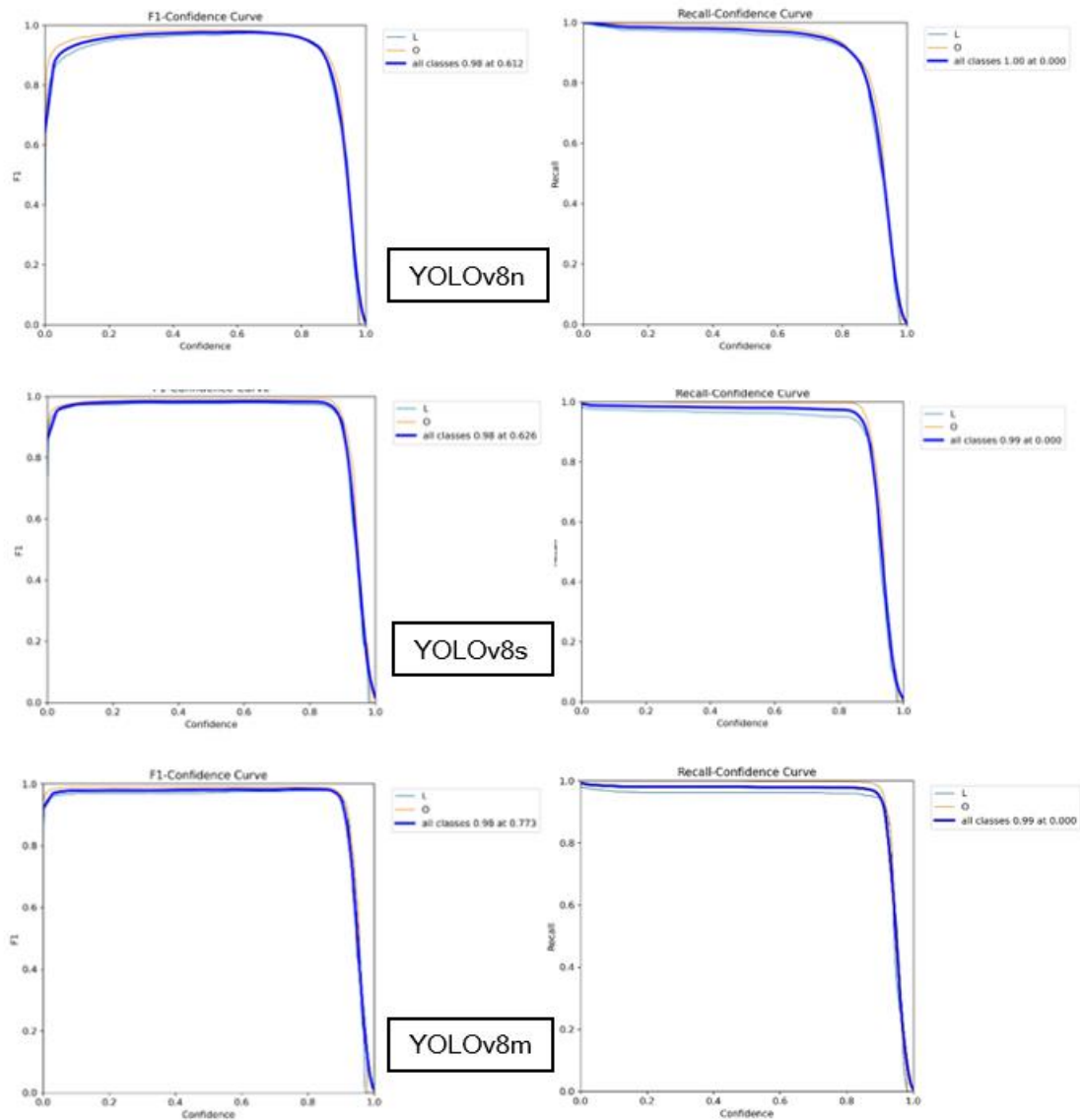


Figura 54: Métricas de F1-Score y Recall de YOLOv8 con dataset "EspaciosUBB"

Ya obtenido el modelo que presenta mejor rendimiento en las versiones cinco y ocho de YOLO, se someten a comparativas para encontrar al que puede funcionar mejor para resolver la problemática del proyecto.

- **Resultados de la mejor arquitectura para la detección de espacios libres y ocupados.**

Tras exhaustivas comparativas, se determina que la versión más eficaz de YOLOv5 es la denominada "m", mientras que en el caso de YOLOv8, la versión óptima es la "m" igualmente. Con los modelos ya seleccionados, se procedió a realizar las últimas evaluaciones para determinar cuál de ellos exhibe el mejor rendimiento y, por tanto, se considera la mejor opción.

Un análisis detallado tanto en la *Figura 55* como en la *Figura 56* reveló que ambos modelos, presentan un comportamiento similar en su entrenamiento, completando las 100 épocas. Y presentando mínimas fluctuaciones, donde convergen rápidamente y se logran estabilizar, indicando un buen proceso de aprendizaje.

Este análisis es muy importante para la toma de decisiones, ya que al observar cual modelo presenta un mejor rendimiento, se destaca levemente YOLOv8m. La elección de este modelo es por su ligero mejor rendimiento a la hora de predecir objetos correctamente, lo que sugiere una mejor capacidad para generalizar y realizar detecciones correctas. Con una precisión del 99.6% respecto a YOLOv5m que presenta un 98.1%.

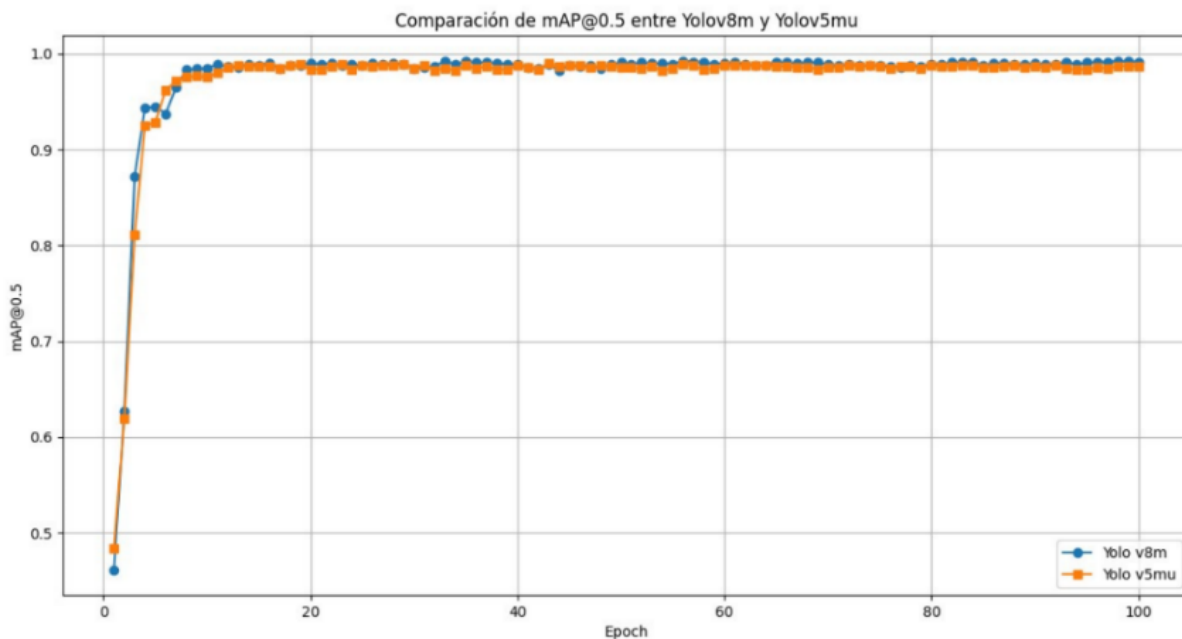


Figura 55: Gráfico comparativo entre YOLOv5m y YOLOv8m

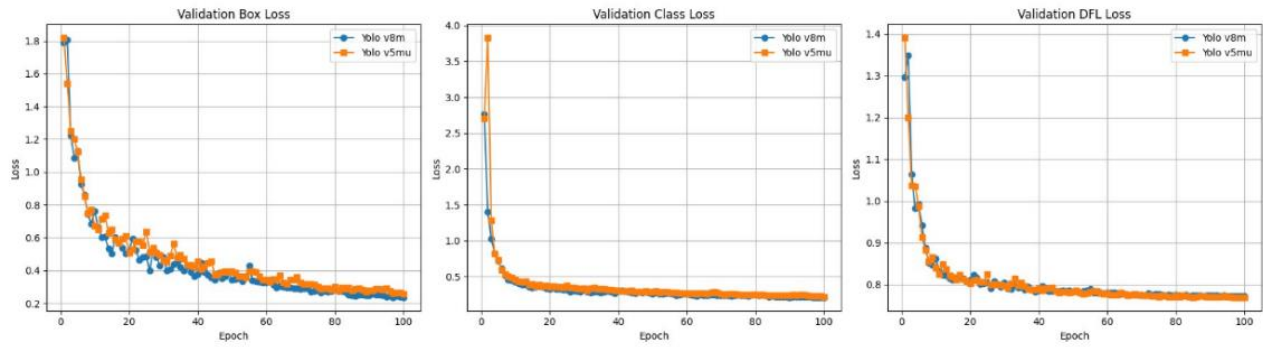


Figura 56: Comparativa entre métricas de evaluación entre YOLOv5m y YOLOv8m

- **Tabla comparativa de resultados de detección de espacios libres y ocupados**

	Métricas	Versión “n”	Versión “s”	Versión “m”
YOLOv5	mAP %	98%	98%	98%
	Precision %	99%	99%	99%
	Recall %	99%	99%	99%
	F1-Score %	97%	98%	98%
YOLOv8	mAP %	99%	99%	99%
	Precision %	99%	99%	100%
	Recall %	99%	99%	99%
	F1-Score %	98%	98%	98%

Tabla 8: Comparativa entre los modelos de YOLO para el dataset EspaciosUBB

Caso 2: Resultados de Dataset “VehículosUBB”

De igual forma, se realiza las respectivas comparaciones entre las subversiones “n” – “s” “m”, de las arquitecturas YOLOv5 y YOLOv8, que han sido entrenados con el dataset que ha sido etiquetado con la clase car, para la detección de vehículos. Esto con el objetivo de encontrar la arquitectura que presente el mejor rendimiento y poder comparar cual es la mejor solución para cumplir el objetivo del proyecto.

- **YOLOv5**

En este caso se usaron las mismas métricas abordadas anteriormente como el mAP, cuya visualización se dispone en la *Figura 57*, la cual muestra el rendimiento de las subversiones “n” - “s” - “m” de Yolov5. El gráfico muestra que el entrenamiento de los modelos era bastante inestable hasta al menos la época 20, lo cual puede ser normal puesto que está aprendiendo, pero posteriormente las líneas parecen estabilizarse, indicando que se estaba convergiendo hacia un rendimiento positivo. Se nota que al final todas las arquitecturas finalizan en diferentes épocas, esto es por la técnica del earlyStopping, que actúa deteniendo el modelo, cuando ya deja de aprender características, esto con tal de evitar el sobreajuste.

Los resultados del gráfico, no se pueden declarar de manera efectiva, puesto que, aunque se muestran resultados prometedores, donde todos parecen superar el 75% de precisión, las líneas presentan fluctuaciones que no permiten asegurar cual modelo es mejor, por ende, es necesario analizar los gráficos de validación.

Estas fluctuaciones pueden deberse a una serie de factores, como, variabilidad en los datos de validación, sobreajuste, entre otros. Aunque se estima que puede deberse al complejo etiquetado que conforman el dataset, que se ha visto limitado por el ángulo de la cámara.

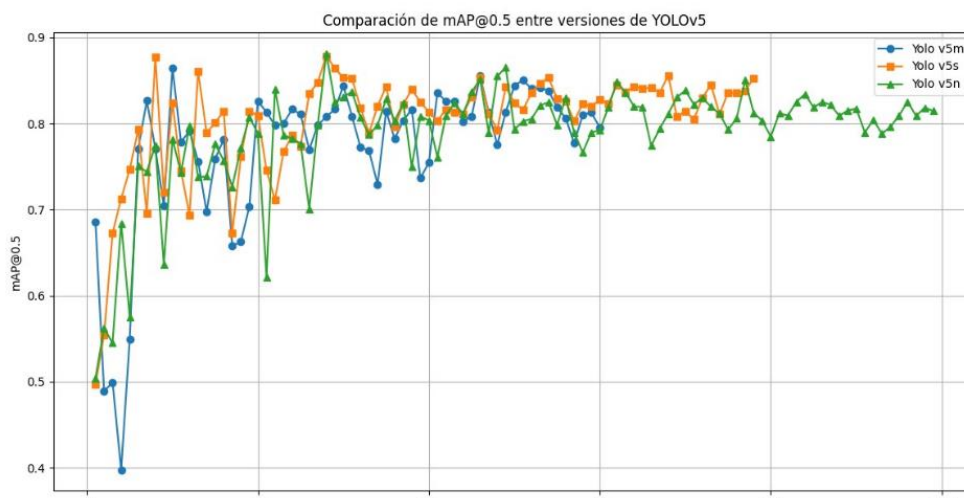


Figura 57: Comparación de mAP de versiones de YOLOv5 con dataset de Vehículos

Al evaluar las métricas de pérdida y comparar los rendimientos según los gráficos de la *Figura 58*, se deduce que la versión “n” y “s” parecen ser similarmente competentes en precisar la ubicación y el tamaño de los bounding box, y así mismo en su rendimiento en la clasificación de objetos dentro de los bounding box, siendo mejores que la versión “m”. Mientras que la versión de YOLOv5s demuestra ser el modelo que mejor ajusta las cajas delimitadoras en términos de superposición y distancia con respecto a los bounding box reales. Siendo esta, la arquitectura que presenta mejores resultados en la comparativa, aunque sin notables diferencias.

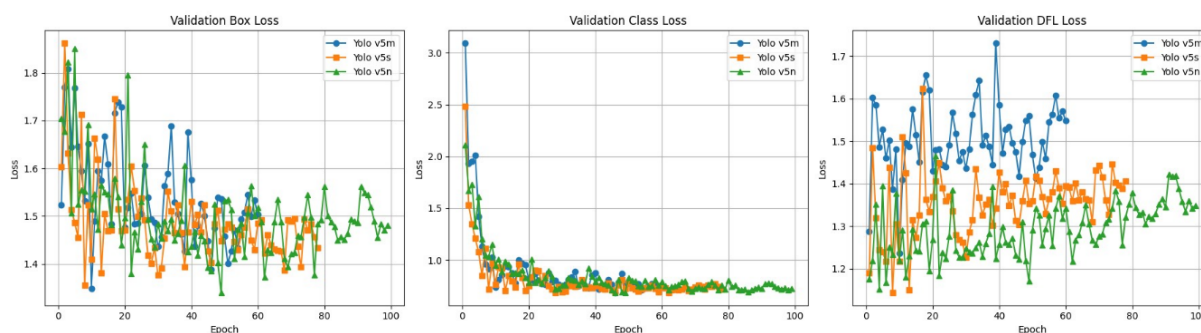


Figura 58: Métricas de validación para entrenamiento de versiones de YOLOv5 con dataset de Vehículos

A continuación, se muestran las matrices de normalización para comprender su precisión, donde se muestra la clase “car” y la clase “Background” que se refiere al fondo de la imagen y donde el modelo puede confundirse. Resultando en la versión YOLOv5s como la mejor con una precisión en detección de vehículos del 88%, y una confusión con el fondo, con una imprecisión del 13%.

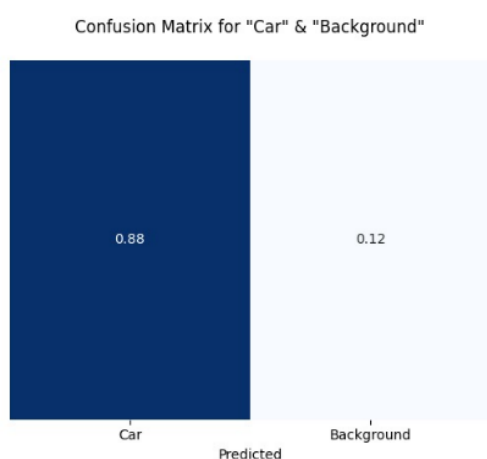


Figura 60: Matriz de confusión normalizada de YOLOv5m

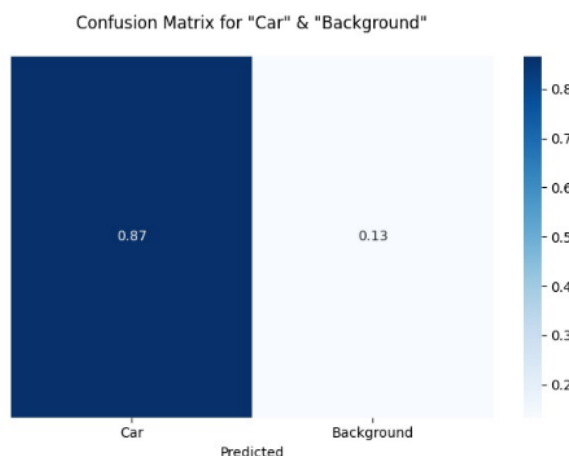


Figura 59: Matriz de confusión normalizada de YOLOv5m

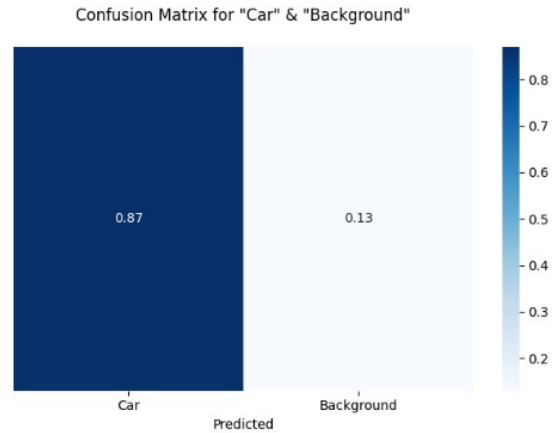


Figura 61: Matriz de confusión normalizada de YOLOv5n

El Recall de los modelos, se muestra en la *Figura 62*, donde se analiza que, en todas las versiones funciona de manera similar. Comenzando con un alto Recall aproximado de 90%, lo que indica que se logra una alta detección de instancias positivas correctamente, con un bajo umbral de decisión. A medida que este umbral sube, el Recall baja de manera gradual hasta una puntuación del 0.9 aproximadamente. Donde finalmente, desciende al tratar las imágenes con un alto índice del umbral de decisión. Concluyendo en las versiones “n” y “m” son ligeramente mejores que la versión “s”, puesto que alcanzan una puntuación de 93% frente a un 91%.

En el F1-Score, se muestra que YOLOv5s, tiene una mayor puntuación a un umbral de confianza ligeramente más bajo respecto a sus rivales que muestran un rendimiento similar. Donde, en el modelo “n”, la métrica alcanzó valores de 83% cuando el umbral era de 0.5, lo cual no es muy efectivo. Mientras que la versión “s” alcanzó un 86% con casi un 0.5 de umbral de confianza, y la versión “m” alcanzó un 85% con una medida del 0.58 umbral de confianza.

Basándose en los factores presentados, se concluye que la mejor opción es el modelo YOLOv5s, por su alta precisión y equilibrio con el Recall, aunque sea con un umbral de confianza despreciablemente más bajo.

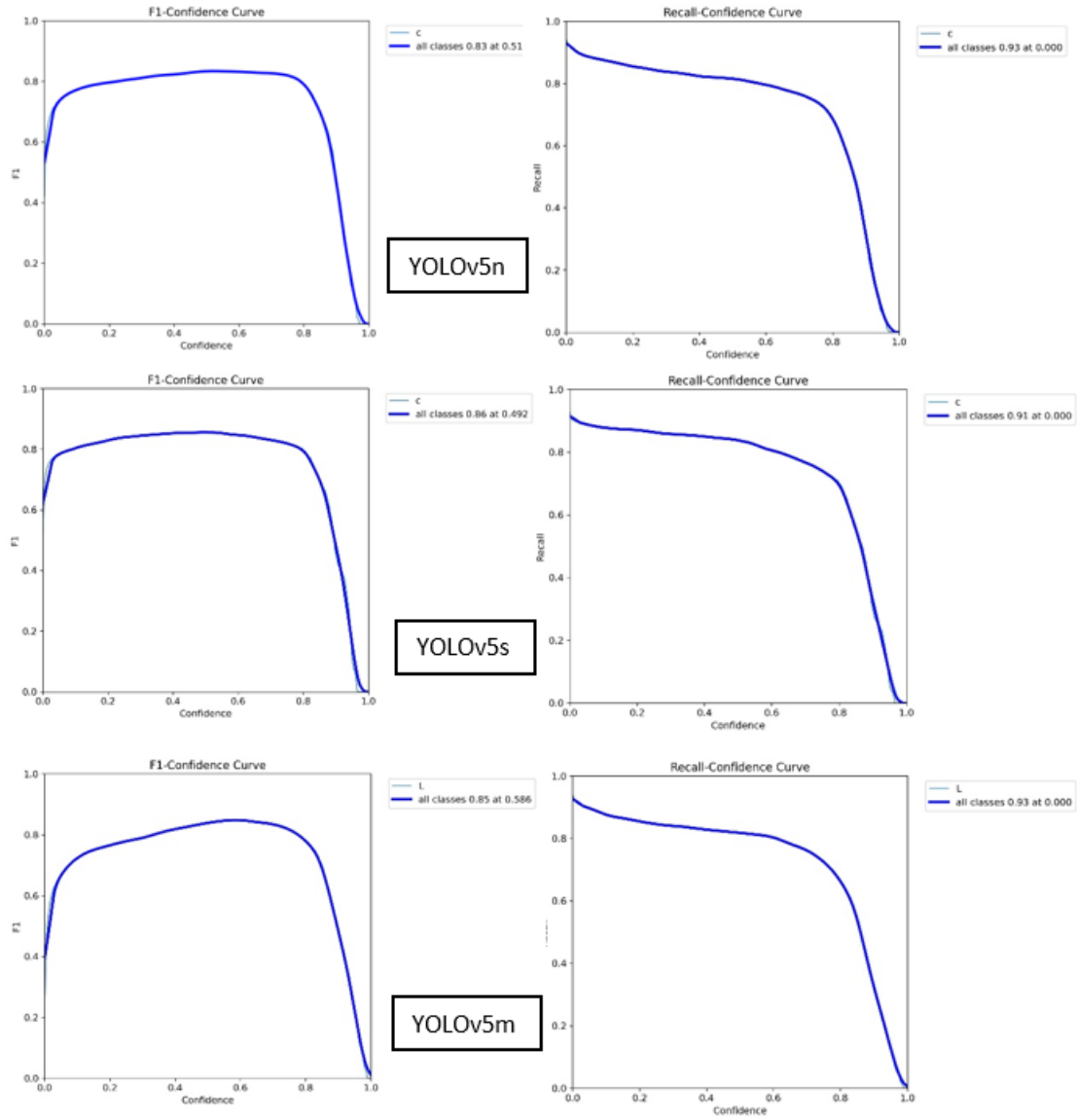


Figura 62: Métricas de F1-Score y Recall de YOLOv5 con dataset "VehículosUBB"

- **YOLOv8**

En el gráfico de la *Figura 63*, se observó que con respecto a la interpretación de YOLOv5, se muestra un mejor rendimiento de manera general, donde a simple vista se destaca que la versión “m” alcanza los mayores índices de precisión media sobre sus rivales, pero sigue teniendo una alta fluctuación que indica poca estabilización y que no generaliza de la mejor manera. Mientras que la versión “s”, aunque no posea la misma precisión, presentó la mejor consistencia y estabilidad.

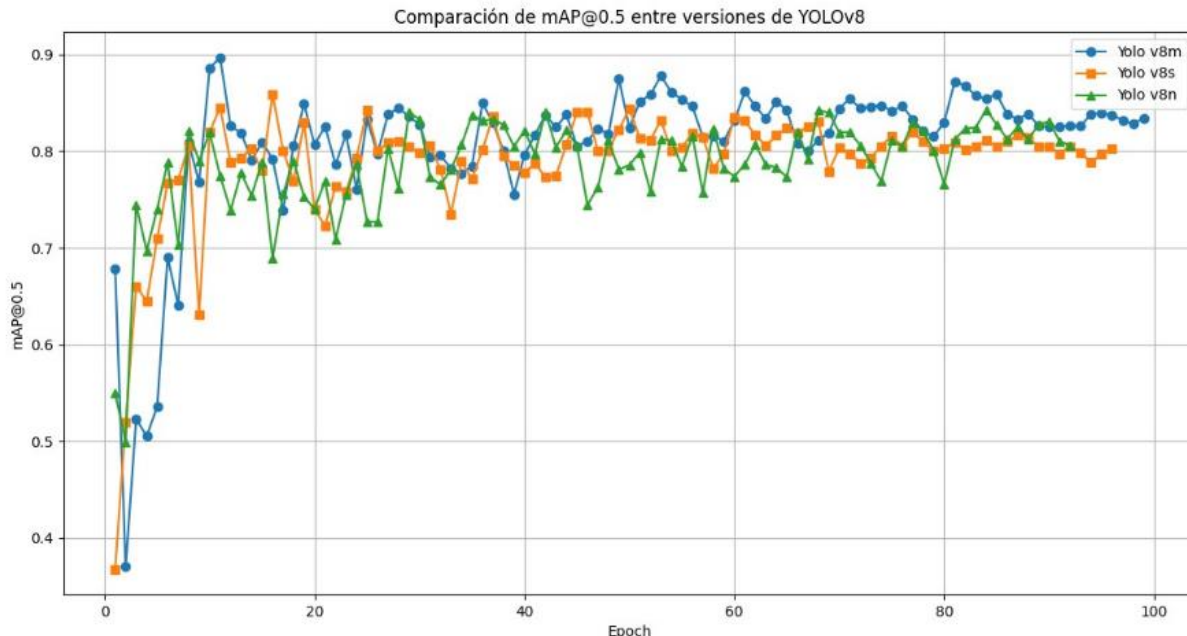


Figura 63: Comparación de mAP de versiones de YOLOv8 con dataset de Vehículos

Respecto a los gráficos de la *Figura 64*, se observa un comportamiento similar al anterior, donde los modelos “s” y “m” parecen mostrar mejor rendimiento, mientras que el “n” presenta demasiada inestabilidad, esto porque al ser una arquitectura menos compleja, le cuesta reconocer los patrones y características de forma fiable, se confunde, no logrando generalizar bien. En cuanto a la pérdida de validación se infiere que la versión “m” de YOLOv8, es la que mejor rendimiento muestra por sobre sus adversarios en el aspecto de precisar el lugar y tamaño de los bounding box para el objeto, mientras que la versión “s” siendo más estabilizada, es la que logra posicionar mejor los bounding box con respecto a los casos reales. Concluyendo en que YOLOv8m es la arquitectura seleccionada.

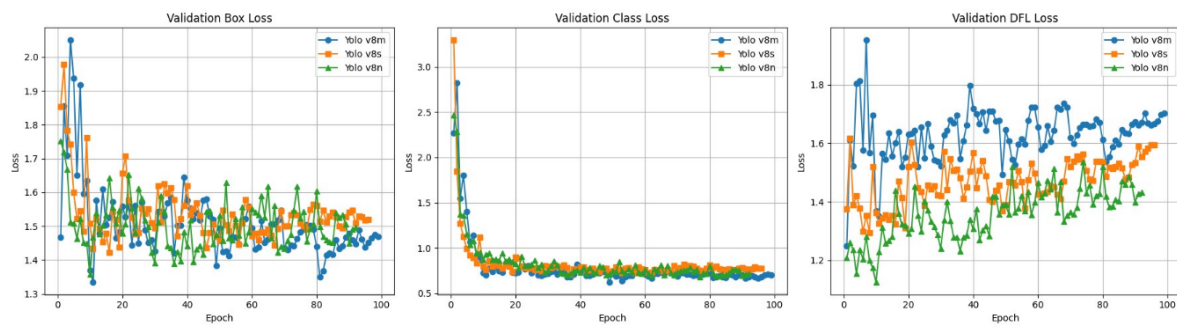


Figura 64: Métricas de validación para entrenamiento de versiones de YOLOv8 con dataset de Vehículos

Las siguientes figuras a continuación muestran las matrices de normalización de los modelos de YOLOv8, para ayudar a visualizar la precisión que tuvieron los modelos. Mostrando una mejor precisión en la versión de la arquitectura YOLOv8m, con un 88% de precisión y un 12% de confusión e imprecisión del modelo con el fondo.

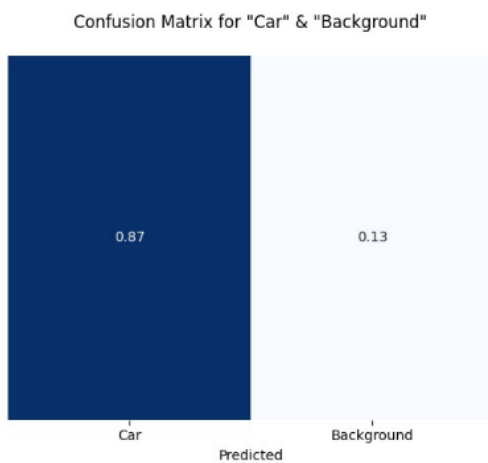


Figura 66: Matriz de Normalización de YOLOv8s

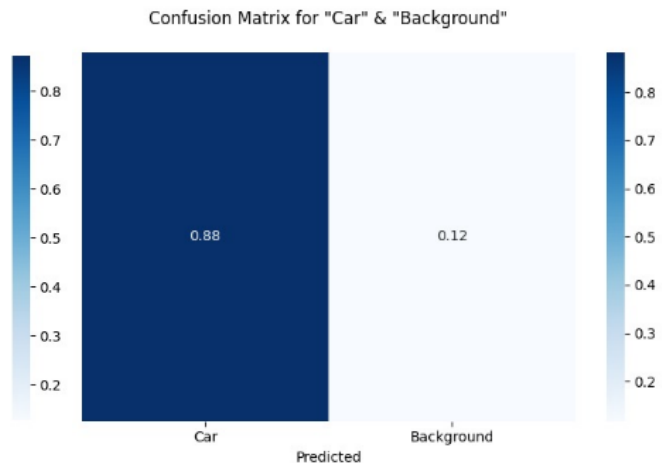


Figura 65: Matriz de Normalización de YOLOv8m

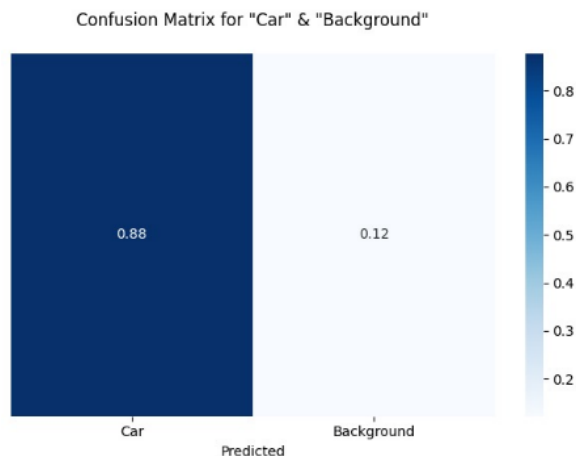


Figura 67: Matriz de Normalización de YOLOv8n

Respecto al Recall de los modelos que se muestra en la *Figura 68*, se infiere que, en todas las versiones muestra resultados similares. Comenzando con un alto Recall aproximado de 90%, lo que indica que se logra una alta detección de instancias positivas correctamente, con un bajo umbral de decisión. Pero a medida que este umbral sube, el Recall baja gradualmente hasta una puntuación del 0.9 aproximadamente. Donde finalmente, desciende considerablemente, al tratar las imágenes con un alto índice del umbral de decisión. Concluyendo en que las versiones "n" muestra un mejor puntaje con 92%, pero apenas mejor que sus rivales "s" y "m", que muestran un 91% de Recall.

En el F1-Score, se muestra que YOLOv5m, tiene una mayor puntuación, pero a un umbral de confianza ligeramente más bajo respecto a sus rivales que muestran un rendimiento similar. Donde, los modelos "n" y "s" alcanzaron valores de 83% cuando el umbral de confianza era de 0.6. Mientras que, la versión "m" alcanzo un 85% con un umbral de confianza cercano al 0.6, pero ligeramente más bajo que sus rivales. Concluyendo en que el mejor modelo es YOLOv8m, por su alto equilibrio entre Recall y precisión.

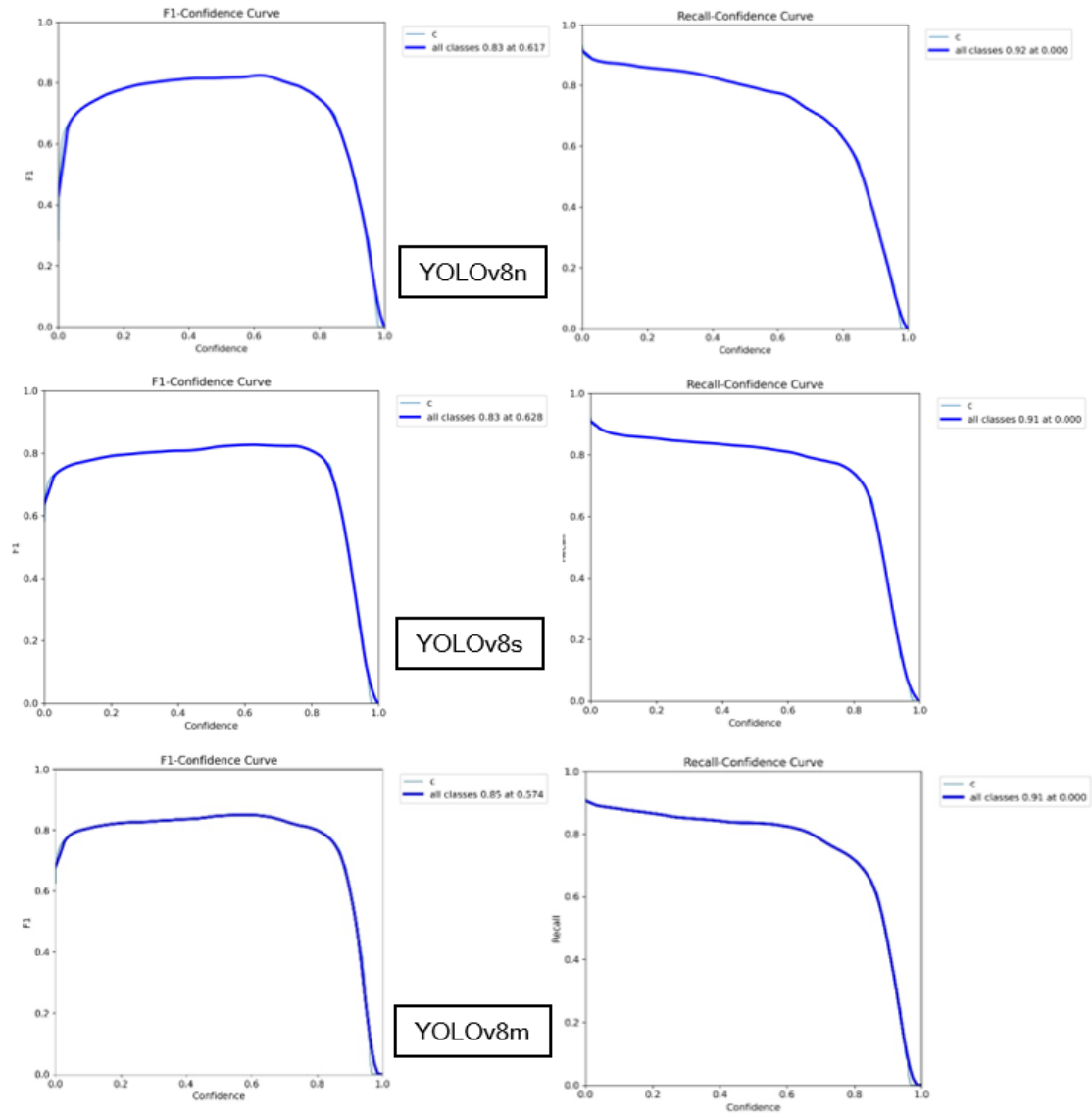


Figura 68: Métricas de F1-Score y Recall de YOLOv8 con dataset "VehículosUBB"

- **Resultados de la mejor arquitectura para la detección de vehículos**

Tras exhaustivas comparativas, se determina que la versión más eficaz de YOLOv5 es la denominada "s", mientras que en el caso de YOLOv8, la versión óptima es la "m". Una vez seleccionados estos modelos, se procedió a las últimas evaluaciones para determinar cuál de ellos exhibe el mejor rendimiento y, por ende, se considera la opción más adecuada.

Un análisis detallado de la *Figura 69* reveló que YOLOv5s interrumpe su proceso de entrenamiento más temprano, específicamente en la época 80, demostrando una mayor estabilidad en comparación con su contraparte. Mientras tanto, YOLOv8m, en su búsqueda de la precisión óptima, completa las 100 épocas, exhibiendo una ligera mejora en términos de precisión.

Este análisis es fundamental para la toma de decisiones, ya que permite sopesar cuidadosamente la estabilidad frente a la precisión en el contexto específico de la detección de objetos en la Universidad del Bío-Bío. La elección de YOLOv5s, con su enfoque en la estabilidad y un rendimiento robusto hasta la época 80, sugiere una mayor confianza en la capacidad del modelo para generalizar y mantener su desempeño en situaciones diversas.

No obstante, se reconoce que la elección entre estabilidad y precisión depende en última instancia de las necesidades específicas de la aplicación. Por lo tanto, este proceso de evaluación proporciona una base sólida para seleccionar la arquitectura que mejor se adapte a los requisitos particulares de detección de espacios en estacionamientos, contribuyendo así a una implementación efectiva y eficiente en la Universidad del Bío-Bío.

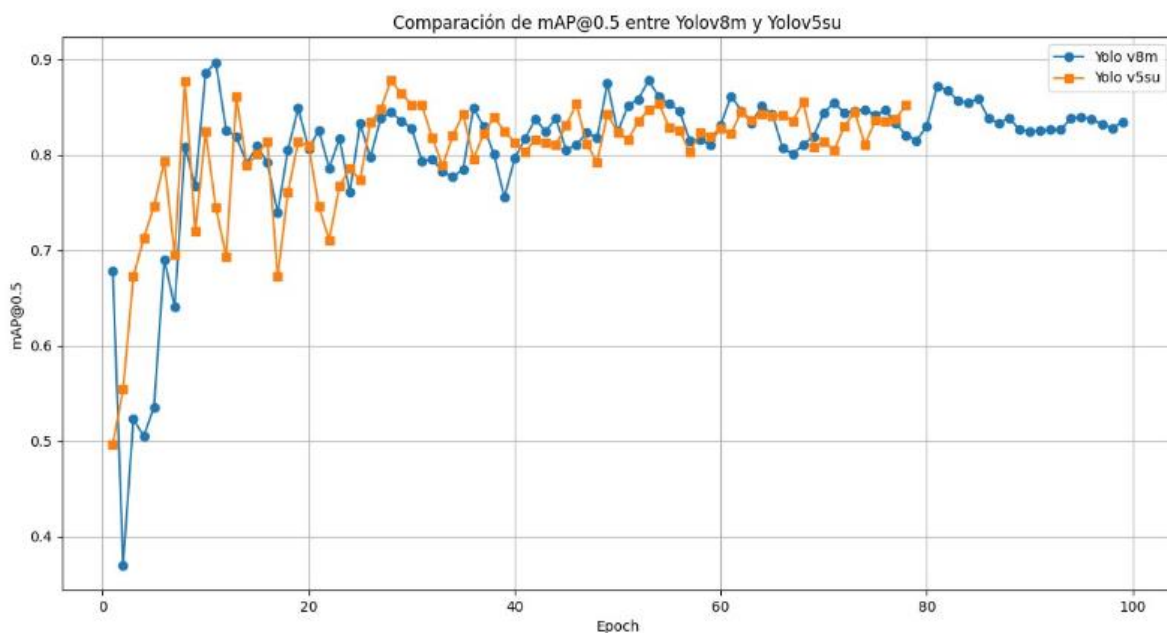


Figura 69: Comparativa del mAP entre YOLOv5s y YOLOv8m

Se define a través de las métricas de pérdida de validación que YOLOv5s presenta mejores cualidades dado su entrenamiento y pruebas, puesto que es capaz de detectar los espacios y clasificarlos de mejor manera que su adversario en la comparación. Se muestra en la *Figura 70*, que esta arquitectura es más estable, logrando generalizar de mejor manera los datos, y posicionando más precisamente los bounding box respecto a los casos reales.

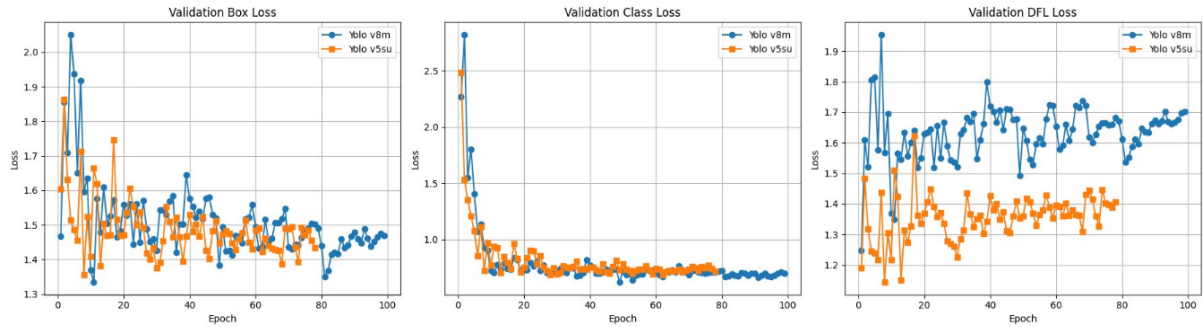


Figura 70: Comparativa de métricas de evaluación entre los modelos YOLOv5s y YOLOv8m

- **Tabla comparativa de resultados de detección de vehículos**

	Métricas	Versión “n”	Versión “s”	Versión “m”
YOLOv5	mAP %	86%	87%	86%
	Precision %	87%	88%	87%
	Recall %	93%	91%	93%
	F1-Score %	83%	86%	85%
YOLOv8	mAP %	84%	84%	87%
	Precision %	88%	87%	87%
	Recall %	92%	91%	91%
	F1-Score %	83%	83%	85%

Tabla 9: Comparativa de los modelos de YOLO para el dataset VehículosUBB

- **Conclusión de las comparaciones para seleccionar el modelo final**

A lo largo de los resultados, se ha llevado a cabo un análisis exhaustivo que revela un desempeño generalmente más sólido de los modelos cuando se emplea el conjunto de

datos "EspaciosUBB" en comparación con el conjunto "VehículosUBB". Esta tendencia se manifiesta en una mayor precisión al reconocer el estado de las plazas de estacionamiento. Es importante destacar que todos los modelos exhiben métricas positivas que superan el umbral del 85% de efectividad, lo cual constituye un resultado alentador y aplicable en el contexto de detección de objetos.

Al concluir el análisis e interpretación de los resultados, se observa que la solución que ha presentado los porcentajes más elevados, tanto en precisión como en las diversas métricas evaluadas, corresponde al escenario en el cual se utilizó el dataset denominado "EspaciosUBB" junto al modelo YOLOv8m. Este modelo específico ha demostrado ser la arquitectura más precisa, destacándose por su capacidad para generalizar de manera efectiva las características de los objetos en las imágenes.

La elección de YOLOv8m como la arquitectura más precisa no solo se basa en la meticulosidad de sus resultados, sino también en su habilidad para adaptarse y generalizar de manera eficaz en diferentes situaciones. Esta precisión y capacidad de generalización son factores críticos para garantizar un rendimiento constante en entornos dinámicos, como los estacionamientos universitarios.

Por lo tanto, se justifica plenamente la elección de implementar el modelo YOLOv8m en la sección siguiente, dado su rendimiento superior y su capacidad para abordar de manera efectiva la problemática específica de la detección de espacios en la Universidad del Bío-Bío.

7.6 Implementación

Etapa 6: En esta etapa se planifica la incorporación del sistema en el entorno objetivo del proyecto, para esto se desarrolla la interfaz web junto a la base de datos que tienen como objetivo poder monitorear de manera continua el modelo ya implementado.

El propósito principal de la aplicación es mantener una comunicación constante entre las predicciones de nuestro modelo, la base de datos y la interfaz de usuario, para esto se utilizó el lenguaje de programación Python con la librería OpenCV, el modelo ya entrenado para la detección de espacios, FastAPI para crear los endpoint que gestionan los datos del estacionamiento, MySQL como base de datos y finalmente Flutter para la interfaz de usuario.

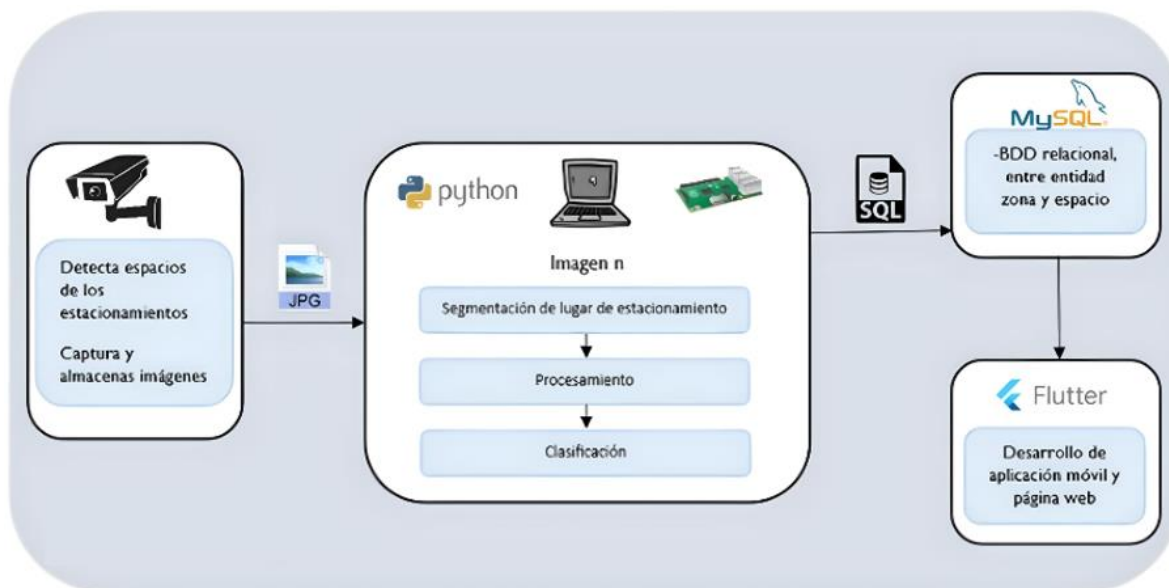


Figura 71: Esquema del funcionamiento del proyecto

En la *Figura 71* se puede apreciar un esquema que representa de manera general el funcionamiento de esta aplicación. En un principio la cámara es responsable de capturar y almacenar las imágenes que se obtendrán mediante video, luego estas imágenes son procesadas por el modelo clasificando los espacios en libres u ocupados, una vez ya clasificados son enviados mediante una api a la base de datos, finalmente la api es la encargada de entregar a la interfaz de usuario los estacionamientos con los espacios actuales.

El primer punto a considerar fue contar con una ubicación precisa de los espacios de estacionamiento, ya que la aplicación depende en gran medida de la exactitud con la identifique y ubique estos datos, además las clasificaciones erróneas pueden llevar a una mala experiencia de usuario.

Para abordar este desafío, se utilizó la técnica ya mencionada anteriormente ROI, las regiones de interés se definieron ubicando las coordenadas en los videos recopilados de la cámara de estacionamiento. En la *Figura 72* se pueden apreciar las coordenadas que detecta el modelo entrenado. Para obtener cada eje dentro de la imagen se utilizó una librería de python llamada PyAutoGUI, la cual tiene la capacidad de convertir el cursor en un visor de coordenadas en tiempo real.

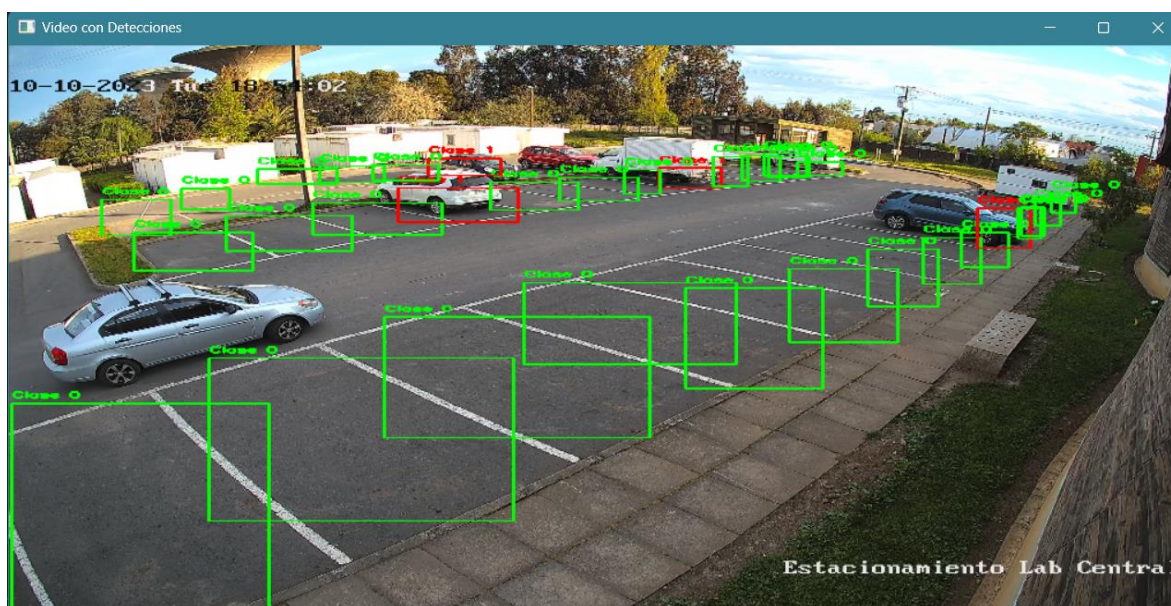


Figura 72: Coordenadas detectadas por el modelo

Finalmente, en la *Figura 73* se pueden apreciar los 7 espacios de estacionamientos utilizados como prueba para llevar a cabo la aplicación.

```

v espacios_fijos = {
    1: {'x1': 0, 'y1': 641, 'x2': 226, 'y2': 1024},
    6: {'x1': 176, 'y1': 553, 'x2': 438, 'y2': 845},
    3: {'x1': 322, 'y1': 479, 'x2': 577, 'y2': 680},
    4: {'x1': 453, 'y1': 392, 'x2': 667, 'y2': 563},
    5: {'x1': 110, 'y1': 329, 'x2': 211, 'y2': 395},
    2: {'x1': 196, 'y1': 302, 'x2': 298, 'y2': 360},
    7: {'x1': 266, 'y1': 280, 'x2': 373, 'y2': 332},
}

```

Figura 73: Coordenadas de ROI en el estacionamiento

El segundo punto a desarrollar fue la escalabilidad del proyecto y la necesidad de utilizar una base de datos para recopilar estadísticas, para esto se optó por MySQL como se puede apreciar en la *Figura 74* junto a su estructura.

				id	espacioID	estado	fechaHora	cantidadVehiculos
☐		Editar		Copiar		Borrar	257	1 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	258	2 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	259	3 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	260	4 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	261	5 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	262	6 Ocupado 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	263	7 Libre 2023-12-05 14:21:18 6
☐		Editar		Copiar		Borrar	264	1 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	265	2 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	266	3 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	267	4 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	268	5 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	269	6 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	270	7 Ocupado 2023-12-05 14:21:46 0
☐		Editar		Copiar		Borrar	271	1 Libre 2023-12-11 22:43:28 6
☐		Editar		Copiar		Borrar	272	2 Libre 2023-12-11 22:43:28 6

Figura 74: Base de datos en MySQL del estacionamiento

Cada estacionamiento tiene un ‘espacioID’ para diferenciarlo de los demás, un ‘estado’ el cual puede tomar el valor de “Libre” u “Ocupado”, una variable nombrada ‘fechaHora’ para facilitar las estadísticas manteniendo los datos históricos y finalmente la cantidad de vehículos totales que se encuentran dentro del estacionamiento en ese momento.

Para manipular el flujo de datos y como estos iban a ser utilizados se diseñaron 2 endpoints con la librería Fastapi, Estos endpoints utilizan los modelos Pydantic ‘SpaceData’ y ‘SpacesData’ los cuales se pueden apreciar en la *Figura 75*

```
class SpaceData(BaseModel):
    espacioID: int
    estado: str
    cantidadVehiculos: int

class SpacesData(BaseModel):
    spaces: List[SpaceData]
```

Figura 75: Creación de endpoints para manipular los espacios

Se definen las estructuras de datos que se enviarán a través de los endpoints creados. En el caso de "SpaceData", esta estructura incluye la inicialización de un identificador (ID), un estado y la cantidad de vehículos en el estacionamiento. Por otro lado, para "SpacesData", la estructura consiste en una lista que contendrá todos los espacios, y estos datos se recibirán de "SpaceData".

Los endpoints en cuestión son dos:

1. `update_spaces` (POST): Este endpoint tiene la responsabilidad de insertar los datos de los espacios de estacionamiento en la base de datos, siguiendo la estructura mencionada anteriormente. Los datos se envían mediante una solicitud de tipo POST.
2. `get_spaces` (GET): Este endpoint se encarga de proporcionar los datos a la interfaz de usuario. En consecuencia, es de tipo GET. Es importante destacar que "get_spaces" obtiene los datos relacionados con la cantidad de espacios de estacionamiento totales, organizados por fecha y hora. Esto se debe a cómo se ha estructurado la base de datos. En lugar de realizar UPDATES, la base de datos inserta todos los espacios de estacionamiento con su estado actual cada vez que ocurre un cambio. Por lo tanto, para garantizar el funcionamiento en tiempo real y mantener un historial de los datos, se ha optado por este enfoque.

Lo mencionado anteriormente se puede apreciar en la *Figura 76*, donde se muestra la base de datos con dos registros. El primero corresponde a la ocupación de los siete estacionamientos en su totalidad, y el segundo indica que el estacionamiento número 6 está ocupado. Además, se puede observar que debido a que el vehículo utilizó el estacionamiento 6 realizó una inserción en el registro de los siete estacionamientos.

☐		Editar		Copiar		Borrar	516	1	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	517	2	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	518	3	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	519	4	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	520	5	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	521	6	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	522	7	Ocupado	2023-12-21 00:27:38	0
☐		Editar		Copiar		Borrar	523	1	Libre	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	524	2	Libre	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	525	3	Libre	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	526	4	Libre	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	527	5	Libre	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	528	6	Ocupado	2023-12-21 00:28:11	6
☐		Editar		Copiar		Borrar	529	7	Libre	2023-12-21 00:28:11	6

Figura 76: Registro Base de Datos

En la *Figura 77* se puede apreciar la aplicación creada en Flutter y cómo esta visualiza y monitorea los espacios de estacionamiento. En este caso se optó por utilizar una imagen satelital de las dependencias para representar de una manera más entendible los estacionamientos y cómo estos se distribuyen. Cada vez que un vehículo ingresa o se desplaza de un espacio, el modelo lo detecta y actualiza los datos, de esta forma la aplicación rescata en tiempo real la cantidad de estacionamientos disponibles consultando al endpoint “get_spaces” explicado anteriormente. Los espacios ocupados se marcan con color rojo, mientras que los disponibles de color verde.

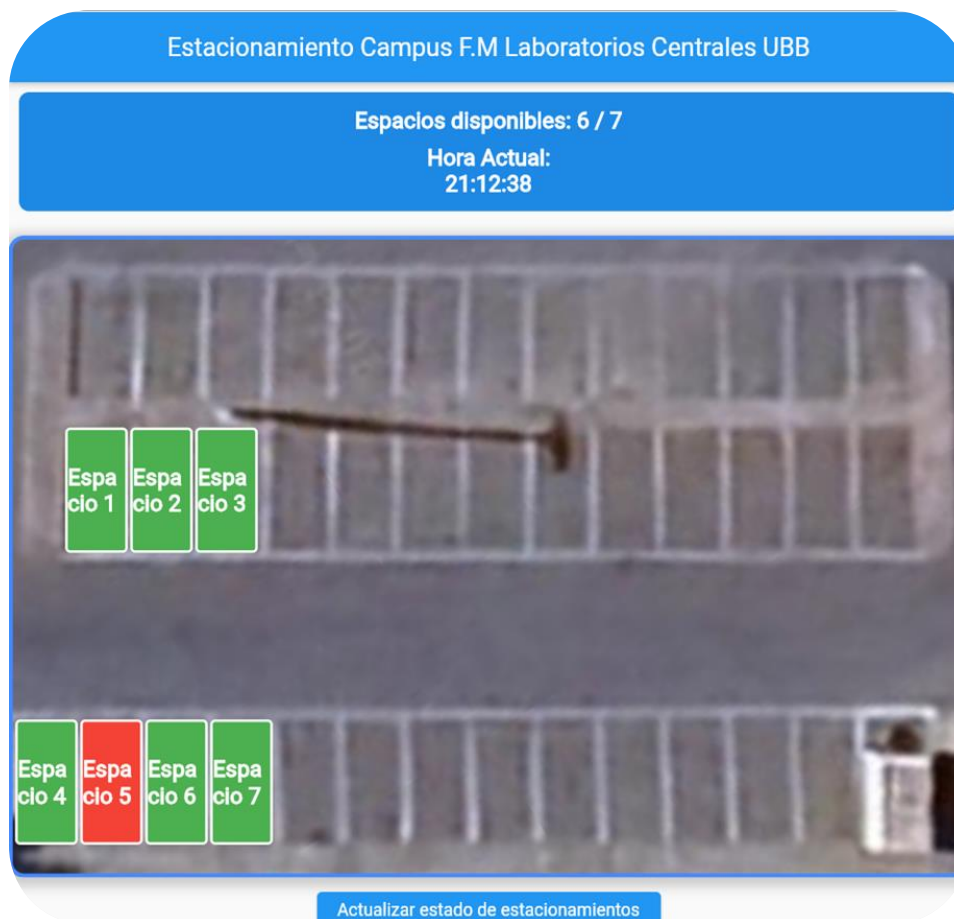


Figura 77: Interfaz de la aplicación para detectar estacionamientos

Finalmente, en la *Figura 78* se puede observar el resultado final con el servidor, el modelo y la interfaz funcionando. En el lado izquierdo está el modelo con cada una de las regiones de interés y al lado derecho la interfaz creada para el usuario con el espacio correctamente marcado como ocupado.

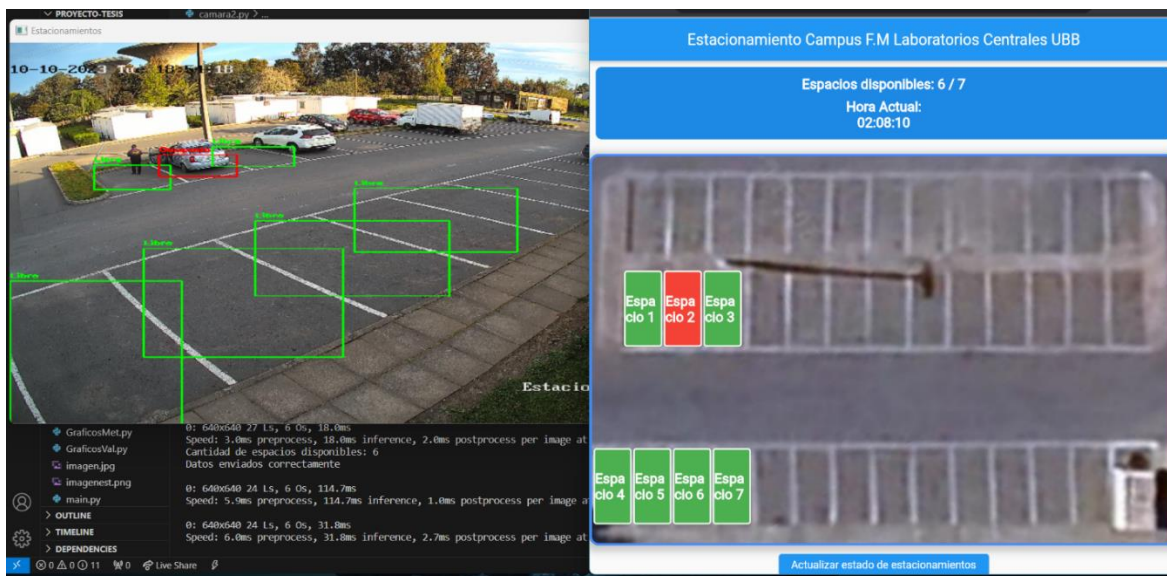


Figura 78: Comparación de la detección de la cámara con la aplicación

8. Conclusiones

A partir del trabajo realizado a lo largo del informe se puede concluir que:

Los objetivos propuestos en el capítulo 3 se llevaron a cabo de manera efectiva, con la selección y comparación de diferentes modelos de visión artificial en dos diferentes conjuntos de datos, donde se seleccionó el de mejor rendimiento para ser utilizado junto a una aplicación. Siendo esta desarrollada exitosamente recogiendo de manera precisa los datos del modelo a través de una base de datos. Permitiendo así la detección de 31 espacios de estacionamiento de los laboratorios centrales en las dependencias de la Universidad del Bío-Bío, Campus Fernando May, Chillán.

Se destaca respecto a otros estudios el hecho de desarrollar una aplicación con una base de datos, lo cual no se había realizado en trabajos anteriores encontrados en la revisión sistemática de la literatura.

Pero lo más destacable y en lo que se centra la investigación, es en la comparativa de diferentes arquitecturas de la red neuronal convolucional YOLO, con un dataset propio. Resultando así, en que el modelo con mejor rendimiento resultó ser la versión YOLOv8m junto al dataset de espacios vacíos y ocupados, mostrando una precisión del 99% en el caso objetivo de la Universidad. Sin embargo, se menciona el hecho de abordar una gran limitante, como lo es el ángulo en que la cámara captura los datos, viéndose solo pequeñas partes de los vehículos en ciertos casos y las líneas del suelo, no siempre se reconocían fácilmente. Donde de los 39 estacionamientos totales, solo se lograron rescatar 31 de manera correcta y quedando 8 espacios que no se pueden distinguir debido al ángulo de la cámara, por ende, no se pudieron ni siquiera etiquetar, siendo esa la razón del 99% de precisión.

De igual manera, el modelo desarrollado, muestra muy buenos índices de efectividad, por lo que se estima que en un entorno con la cámara en una posición donde capture a los vehículos sin interceptarse a tal nivel, se obtendrían resultados perfectos de todo el estacionamiento en su completitud. Siendo el mejor ángulo desde atrás o desde el frente de los objetos.

El desarrollo de este proyecto de investigación aplicada muestra resolver la problemática con índices positivos como se había planteado, lo cual, si se implementa a nivel macro, puede llegar a ser una gran ayuda para agilizar el proceso de estacionar a los usuarios, además de resolver otros problemas indirectamente, como ambientales, económicos, y psicológicos.

8.1 Limitaciones

Aunque el desarrollo del modelo muestra resultados relativamente positivos, en ciertos aspectos presenta limitaciones que impiden un resultado óptimo e ideal del modelo, siendo los siguientes puntos:

- La falta de visión de la totalidad del estacionamiento, producto del limitado ángulo que presenta la cámara respecto al espacio de estacionamiento, donde autos que se encuentran más cercanos al dispositivo, obstruyen la visión de este para observar los vehículos que se encuentren más alejados. Produciéndose así solapamiento de unos vehículos a otros, lo que impide detectar lugares donde existen espacios en que puede haber objetos.
- La falta de cámaras en las otras zonas de estacionamiento de la Universidad, para haber obtenido más datos para el dataset, y así diversificarlo aún más. Y tener la oportunidad de probar el modelo en otros entornos.
- La falta de acceso a la información que captura la cámara en tiempo real, por la forma en que está constituido el sistema de vigilancia, denegando la accesibilidad a terceros.
- La obtención de un dron que permita tomar fotografías aéreas de los estacionamientos de la Universidad. Esto para que luego en la aplicación, se use de fondo para referenciar el espacio real de donde se informa al usuario, y poder crear una interfaz llamativa e intuitiva.
- Y la falta de poder computacional, al contar con un notebook que, aunque presenta características técnicas mayores a la media, no lograba ejecutar ciertas versiones de la arquitectura por su gran complejidad y cantidad de parámetros, estando restringido por su memoria.

9. Trabajos Futuros

Se proponen ciertos puntos para continuar con el desarrollo de este proyecto, y para mejorar el modelo obtenido, como:

- A largo plazo, producto de la instauración del proyecto de seguridad y televigilancia de la Universidad, se espera la implementación de más cámaras de seguridad en las distintas zonas de estacionamiento, por lo cual se plantea expandir el proyecto a lo largo de las diferentes zonas dentro de la Universidad. Donde se recomienda que la instalación de las cámaras presente un ángulo adecuado hacia la captura de los vehículos, donde no se solapen unos a otros.
- Se propone la captura de imágenes de estacionamientos en las distintas épocas del año, para aumentar así su diversidad, y mejorar su rendimiento en variadas situaciones, al entrenarlo con estos datos. Especialmente enfocarse en el invierno con las lluvias torrenciales y los días con neblina.
- Un punto importante, es la mejora de la aplicación, que se encarga de transmitir los datos detectados por el modelo, a los usuarios, que son quienes usan los estacionamientos. Estas mejoras radican en el aspecto de visualización de los datos, que sea una interfaz intuitiva que muestre a los usuarios, donde hay espacios disponibles para aparcar de manera sencilla y rápida.
- Se planteó desde un comienzo, la idea de tener una especie de pantalla en la entrada de vehículos de la Universidad, donde se muestre la cantidad de espacios disponibles y ocupados. Así el usuario sabrá de inmediato donde puede dejar su vehículo y no perder tiempo en búsquedas. Además, esto es mucho más cómodo, ya que no necesita que las personas descarguen una aplicación y vayan observando el dispositivo móvil para obtener la información.

10. Referencias

About. (2019, marzo 6). OpenCV. <https://opencv.org/about/>

Autocasión. (2013, diciembre 17). Buscar aparcamiento, lo que más estresa a los conductores. Autocasión. <https://www.autocasion.com/actualidad/noticias/la-busqueda-de-aparcamiento-actividad-mas-estresante-para-los-conductores-espanoles>

Dart programming language. (s/f). Dart.dev. Recuperado el 14 de diciembre de 2023, de <https://dart.dev/>

Deep Learning o Aprendizaje profundo : ¿qué es? (2022, abril 19). Formation Data Science | Datascientest.com. <https://datascientest.com/es/deep-learning-definicion>

Díaz, R. (2020, mayo 8). Métricas de Clasificación. The Machine Learners. <https://www.themachinelearners.com/metricas-de-clasificacion/>

Duy-Linh Nguyen, Xuan-ThuyVo, Adri Priadana, Kang-Hyun Jo. (2023, agosto 1). YOLO5PKLot: A Parking Lot Detection Network Based on Improved YOLOv5 for Smart Parking Management System - researchr publication authors. Researchr.org. <https://researchr.org/publication/NguyenVPJ23/authors>

El tutorial de Python. (s/f). Python documentation. Recuperado el 14 de diciembre de 2023, de <https://docs.python.org/es/3/tutorial/>

FAQ. (s/f). Flutter.dev. Recuperado el 14 de diciembre de 2023, de <https://docs.flutter.dev/resources/faq>

Fernandez Fernandez, J. M., & Florez Lopez, R. (2008). Las Redes Neuronales Artificiales. Netbiblo.

Google colab. (s/f). Google.com. Recuperado el 14 de diciembre de 2023, de <https://research.google.com/colaboratory/faq.html>

Gutiérrez, C. (2005). Researchgate.net. https://www.researchgate.net/publication/251671565_Analisis_y_revision_de_la_literatura_en_el_contexto_de_proyectos_de_fin_de_carrera_Una_propuesta

Haya, P. (2021, noviembre 29). La metodología CRISP-DM en ciencia de datos - IIC. Instituto de Ingeniería del Conocimiento. <https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/>

Jay. (2023, mayo 6). Introducción a YOLO: detección de objetos en tiempo real. HashDork. <https://hashdork.com/es/yolo/>

LabelImg. (s/f). PyPI. Recuperado el 14 de diciembre de 2023, de <https://pypi.org/project/labelImg/1.4.0/>

Machine Learning: definición, funcionamiento, usos. (2021, diciembre 13). Formation Data Science | Datascientest.com. <https://datascientest.com/es/machine-learning-definicion-funcionamiento-usos>

Matplotlib — visualization with python. (s/f). Matplotlib.org. Recuperado el 14 de diciembre de 2023, de <https://matplotlib.org/>

Max pooling. (s/f). Paperswithcode.com. Recuperado el 14 de diciembre de 2023, de <https://paperswithcode.com/method/max-pooling>

Nguyen, T., Tran, T., Mai, T., Le, H., Le, C., Pham, D., & Phung, K.-H. (2021). An adaptive vision-based outdoor car parking lot monitoring system. 2020 IEEE Eighth International Conference on Communications and Electronics (ICCE).

pandas - Python Data Analysis Library. (s/f). Pydata.org. Recuperado el 14 de diciembre de 2023, de <https://pandas.pydata.org/about/>

Psicocode. (2022, junio 8). El tiempo en buscar aparcamiento afecta a la salud. Psicocode. <https://psicocode.com/miscelanea/tiempo-buscar-aparcamiento/>

¿Qué son las redes neuronales convolucionales? (s/f). IBM.com. Recuperado el 14 de diciembre de 2023, de <https://www.ibm.com/es-es/topics/convolutional-neural-networks>

Rahman, S., Ramli, M., Arnia, F., Sembiring, A., & Muharar, R. (2020). Convolutional neural network customization for parking occupancy detection. 2020 International Conference on Electrical Engineering and Informatics (ICELTICs).

Rusell, S., & Norvig, P. (2004). Inteligencia Artificial Un Enfoque Moderno segunda edición.

Sara, E. (2020, abril 13). La pesadilla de buscar estacionamiento, sobre la mesa. Moviliblog. <https://blogs.iadb.org/transporte/es/la-pesadilla-de-buscar-estacionamiento-sobre-la-mesa/>

SoyChile. (2023). Desde que Ñuble es región, el parque automotriz aumentó un 23%. <https://www.soychile.cl/Chillan/Sociedad/2023/03/22/804394/parque-automotriz-aumento-23-nuble.html>

Thomas, T., & Bhatt, T. (2018). Smart car parking system using convolutional neural network. 2018 International Conference on Inventive Research in Computing Applications (ICIRCA).

Ultralytics. (s/f). Ultralytics.com. Recuperado el 14 de diciembre de 2023, de <https://www.ultralytics.com/es/>

Usach, D. (s/f). Parque automotriz en Chile creció 10% desde la prepandemia y superó los 6 millones. *Diario Usach*. Recuperado el 14 de diciembre de 2023, de <https://www.diariousach.cl/parque-automotriz-en-chile-crecio-10-desde-la-prepandemia-y-supero-los-6>

Viñuela, S. (2015, abril 16). Pierdes 216 euros al año por buscar aparcamiento. *Auto Bild España*. <https://www.autobild.es/noticias/los-espanoles-pierden-dinero-buscando-aparcamiento-251077>

Weiling Chen, C. K. Y. (2019, agosto 1). Unauthorized Parking Detection using Deep Networks at Real Time. *ieeexplore.ieee.org*. <https://ieeexplore.ieee.org/document/8783992>

What is Computer Vision? (s/f). *Ibm.com*. Recuperado el 14 de diciembre de 2023, de <https://www.ibm.com/es-es/topics/computer-vision>

What is NumPy? — NumPy v2.0.dev0 Manual. (s/f). *Numpy.org*. Recuperado el 14 de diciembre de 2023, de <https://numpy.org/devdocs/user/whatisnumpy.html>

Zhang, W., Yan, J., & Yu, C. (2019). Smart parking system based on convolutional neural network models. 2019 6th International Conference on Information Science and Control Engineering (ICISCE).

(S/f-a). *Zamoranews.com*. Recuperado el 14 de diciembre de 2023, de <https://www.zamoranews.com/articulo/reportajes/buscar-aparcamiento-factor-estres-mitad-conductores/20210615114136167166.html>

(S/f-b). *Pytorch.org*. Recuperado el 14 de diciembre de 2023, de <https://pytorch.org/>

(S/f-c). *Mysql.com*. Recuperado el 14 de diciembre de 2023, de <https://www.mysql.com/about/>

(S/f-d). *Visualstudio.com*. Recuperado el 14 de diciembre de 2023, de <https://code.visualstudio.com/docs/editor/whyvscode>

11. Anexo

Repositorio

Para acceder al código fuente y los archivos relacionados con este proyecto, consulte el repositorio en línea en GitHub:

Carrasco R, Rivera, J. (2023). "UBBSmartParking". GitHub. <https://github.com/ICI-UBB/UBBSmartParking>.

UBBSmartParking Public		Edit Pins	Watch 1
main	1 Branch	0 Tags	Go to file
Add file		<> Code	
Mckngbird final		5184a94 · yesterday	9 Commits
app	Proyecto-tesis	yesterday	
scriptbdd	bdd	yesterday	
.gitattributes	Initial commit	yesterday	
README.md	final	yesterday	
best.pt	correcciones	yesterday	
main.py	correcciones	yesterday	
modelo.py	final	yesterday	
parking_space_detector.py	correcciones	yesterday	
prueba2.mp4	correcciones	yesterday	
prueba4.mp4	correcciones	yesterday	
requirements.txt	Proyecto-tesis	yesterday	